INNOPOLIS UNIVERSITY

SNE MASTER
TRACK 2026

COURSE: [S26] Secure Development

Lab 4 Report

On

SIEM

Student Name: HASIB AL TAHSIN

Application ID: 187623
Student ID: 24

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 20.03.2026

Task 1

a. Give a brief explanation of the architecture of your SIEM solution.

Solution:

We have deployed an open-source SIEM (Security Information and Event Management) solution called Wazuh in the lab. Its architecture is essentially distributed and scalable, working with four main components:

1. **Wazuh Agent (The Data Collector):** This small software is installed on the target machine (e.g. Linux 'VM-K8S' and Windows host). It uses the 'Logcollector' and 'Syscheck' (FIM) modules in the background to collect system events, authentication logs (`auth.log`, `access.log`) and real-time file changes (using the OS's inotify). It then sends this data to the manager over an encrypted channel.

2. Wazuh Server / Manager (The Brain): This is the main center of data processing. After receiving the raw data from the agent, the manager uses its decoders to parse the logs (e.g., IP, User, Action). The Rule Engine then analyzes the parsed data. If a threat (e.g., Rule 5402 - Successful sudo to ROOT, or Rule 554 - New file added) is matched, it generates an alert and triggers an Active Response (e.g., blocking the malicious IP) as needed.

3. Wazuh Indexer (The Storage & Search Engine): This is a high-performance database built on top of OpenSearch. The security alerts ('alerts.json') created by the manager are stored and indexed here, so that I can later perform rapid searches and analyses on the large dataset.

4. Wazuh Dashboard (The UI): This is a web-based interface. We visualize the data collected by the indexer through this dashboard. The lab's threat hunting, log filtering, and graphical reports of security events are mainly monitored from this part.

b. Provide 3 advantages of open source solutions and how do these vendors actually make money?

Solution:

Three Advantages of Open-Source Solutions:

Cost-Effectiveness: Commercial or proprietary SIEM tools (such as Splunk or QRadar) come with hefty licensing and data volume costs. Wazuh is completely free, allowing me to use premium features like FIM, Threat Detection, and Active Response in the lab without any enterprise license fees.

Deep Customization & Transparency: The open-source code makes it possible to audit the architecture of the system, eliminating the risk of hidden backdoors. It is also very flexible; for example, in the lab, we directly edited the agent's 'ossec.conf' file to monitor a specific directory ('/root') of our choice, which is not easily done with closed-source tools.

Community-Driven Threat Intelligence: Because it is open-source, thousands of security experts around the world are constantly updating its ruleset and decoders. As a result, when a new cyber attack occurs, its detection rules are quickly available through the community and are mapped beautifully to the MITRE ATT&CK framework.

How do these vendors actually make money:

Although the software is free, open-source vendors (such as Wazuh Inc.) primarily generate revenue through the following methods:

1. Managed Cloud Hosting (SaaS): Large organizations don't want to take on the hassle of maintaining and scaling their own servers. So they use hosted solutions in the vendor's "Cloud Environment," for which the vendor charges a monthly or annual subscription fee based on data retention and compute power.

2. Enterprise Support & SLA: Corporate organizations sign paid SLA (Service Level Agreement) with vendors for 24/7 guaranteed and priority technical support in the event of a system down or a complex cyber attack in a critical production environment.

3. Professional Services & Training: Vendors also earn revenue by selling official cybersecurity training, professional certification courses, and custom third-party integration (such as specific API development) services.

Task 2 - Setup infrastructure

a. Configure a SIEM solution with 3(or more) unique devices. e.g Windows, Linux and a Network device. Can you view log data from each connected device? If yes show this.

Solution:

Manager Installation & Troubleshooting

Several technical challenges (especially port conflicts) were encountered while setting up the Wazuh SIEM platform on the deploy-24 Linux machine. Below is a detailed analysis of them:

A. Port Conflict Resolution

The following command checks which processes are running in the background on the system and are occupying port 9200.

```
hasib@deploy-24:~$ sudo ss -tulpn | grep :9200
tcp   LISTEN 0      4096               0.0.0.0:9200       0.0.0.0:*    users:(("docker-proxy",pid=40346,fd=8))
tcp   LISTEN 0      4096                  [::]:9200          [::]:*    users:(("docker-proxy",pid=40353,fd=8))
```

The above command uses ss: Socket Statistics, a modern tool for viewing network connections. -tulpn: t (TCP), u (UDP), l (Listening ports), p (Process name), n (Numeric address). | grep :9200: Filters out only the line for port 9200 from the entire result.

Checking Docker Processes:

Since some work was already running on the Manager machine, the Docker container was checked.

```
hasib@deploy-24:~$ sudo docker ps
CONTAINER ID   IMAGE                                        COMMAND                  CREATED       STATUS       PORTS
                  NAMES
a5e61b7b9f94   opensearchproject/opensearch-dashboards:2     "./opensearch-dashbo…"   2 days ago    Up 2 days    0.0.0.0:5601->5601/tcp, [::]:5601->5601/tcp
                  os-dashboards
f97acd95faa3   opensearchproject/opensearch:2               "./opensearch-docker…"   2 days ago    Up 2 days    9200/tcp, 9300/tcp, 9600/tcp, 9650/tcp
                  os-node3
8d51f8a38d76   opensearchproject/opensearch:2               "./opensearch-docker…"   2 days ago    Up 2 days    9200/tcp, 9300/tcp, 9600/tcp, 9650/tcp
                  os-node2
f58a081df0b0   opensearchproject/data-prepper:2             "bin/data-prepper"       2 days ago    Up 2 days    0.0.0.0:21893->21893/tcp, [::]:21893->21893/tcp
                  data-prepper
5f5c4835b89d   opensearchproject/opensearch:2               "./opensearch-docker…"   2 days ago    Up 2 days    9300/tcp, 9600/tcp, 0.0.0.0:9200->9200/tcp, [::]:9200
->9200/tcp, 9650/tcp   os-node1
1b9317b76f67   luffysungod/24-webapp:latest                 "python backend/main…"   13 days ago   Up 2 days    0.0.0.0:8080->5000/tcp, [::]:8080->5000/tcp
                  24-webapp
```

This command lists all currently running (Active) Docker containers. The output shows that some containers like opensearchproject and 24-webapp are running, which occupy ports 9200, 9300, 9600, etc. Wazuh itself uses a custom version of OpenSearch (Wazuh Indexer), so Wazuh could not be installed without stopping these containers.

Freeing up the Port:

With this command, the containers that were occupying the port are specifically stopped by their CONTAINER ID.

```
hasib@deploy-24:~$ sudo docker stop a5e61b7b9f94 f97acd95faa3 8d51f8a38d76 f58a081df0b0 5f5c4835b89d 1b9317b76f67
a5e61b7b9f94
f97acd95faa3
8d51f8a38d76
f58a081df0b0
5f5c4835b89d
1b9317b76f67
```

Then run the following command again to confirm that port 9200 is now completely free and ready for Wazuh installation.

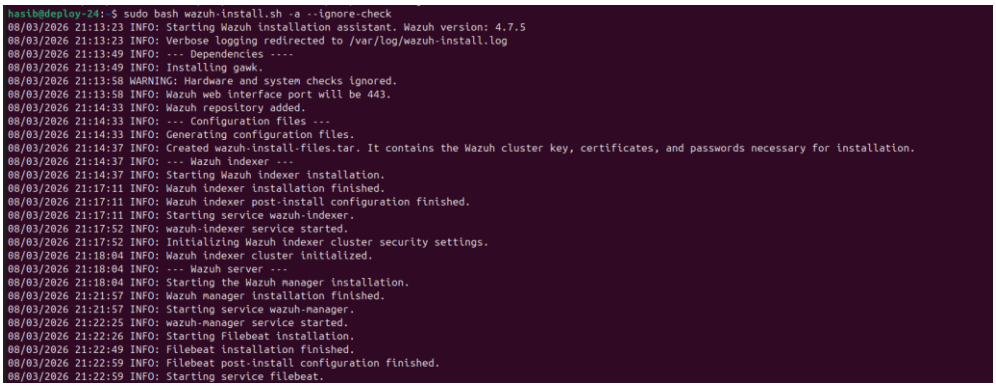
```
hasib@deploy-24:~$ sudo ss -tulpn | grep :9200
```

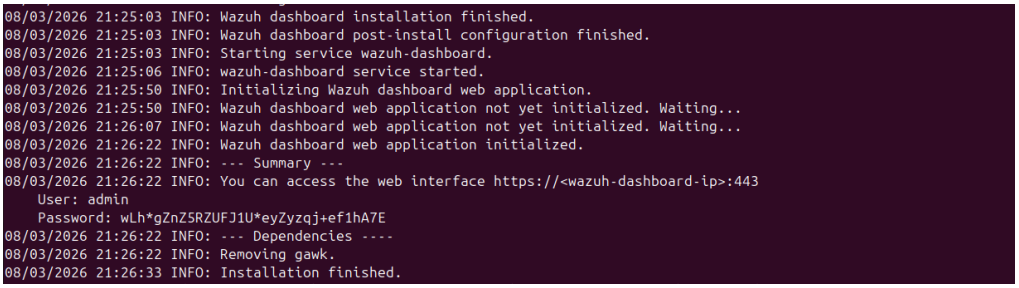
B. Wazuh Manager Installation

After the ports are freed, Wazuh's all-in-one architecture is installed.

Running the installation script:

The official automated installation of Wazuh is done with the following command.





Here -a: means "All-in-one". This means that the three main parts of Wazuh (Indexer, Manager, and Dashboard) are installed on the same server. Then --ignore-check: bypasses hardware checks to ensure that the script does not stop if there are any warnings about RAM or storage on the system.

Password Extraction:

After installation, the system automatically generates some very secure passwords for all services and saves them in a tar file. The following command prints only the password text file directly from the zip file to the terminal, instead of unzipping the entire zip file.

Command:

```
hasib@deploy-24:~$ sudo tar -axf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt -O
[sudo] password for hasib:
# Admin user for the web user interface and Wazuh indexer. Use this user to log in to Wazuh dashboard
  indexer_username: 'admin'
  indexer_password: 'wLh*gZnZ5RZUFJ1U*eyZyzqj+ef1hA7E'

# Wazuh dashboard user for establishing the connection with Wazuh indexer
  indexer_username: 'kibanaserver'
  indexer_password: 'z4*+Nf*m+aul3BlkoNyhsyX+ef2qYjML'

# Regular Dashboard user, only has read permissions to all indices and all permissions on the .kibana index
  indexer_username: 'kibanaro'
  indexer_password: 'jJiw52+Bj4S8TFsoM+eZoAUn75jH1s8+'

# Filebeat user for CRUD operations on Wazuh indices
  indexer_username: 'logstash'
  indexer_password: '5egMZW0XV+WlvPAW5.HxuCm1JPBRIzNe'

# User with READ access to all indices
  indexer_username: 'readall'
  indexer_password: '*Jgo3mU4hkv.LIm?YZe*ykKgkNxkpzHv'

# User with permissions to perform snapshot and restore operations
  indexer_username: 'snapshotrestore'
  indexer_password: '6zB574bv8J1FI*43DnCIdGmAMuuByrR+'

# Password for wazuh API user
  api_username: 'wazuh'
  api_password: 'Qh.BZXaTAoFEqKrjz1k4k*TYWuf5Y0ro'
```

```
# Password for wazuh API user
  api_username: 'wazuh'
  api_password: 'Qh.BZXaTAoFEqKrjz1k4k*TYWuf5Y0ro'

# Password for wazuh-wui API user
  api_username: 'wazuh-wui'
  api_password: '97LJiyREg4C.R2CUqoWmjVY9lRqKbGjf'
```

-axf means auto detect format through a, extract by x, f means file name. -O: (Capital O) means "Extract to Standard Output". That is, the file is displayed directly on the screen without saving it to the hard disk. In the output, I got the password of the admin user, with which I will login to the web dashboard. In addition, the password of kibanaserver, logstash, and wazuh API has also been generated, which is used for backend communication.

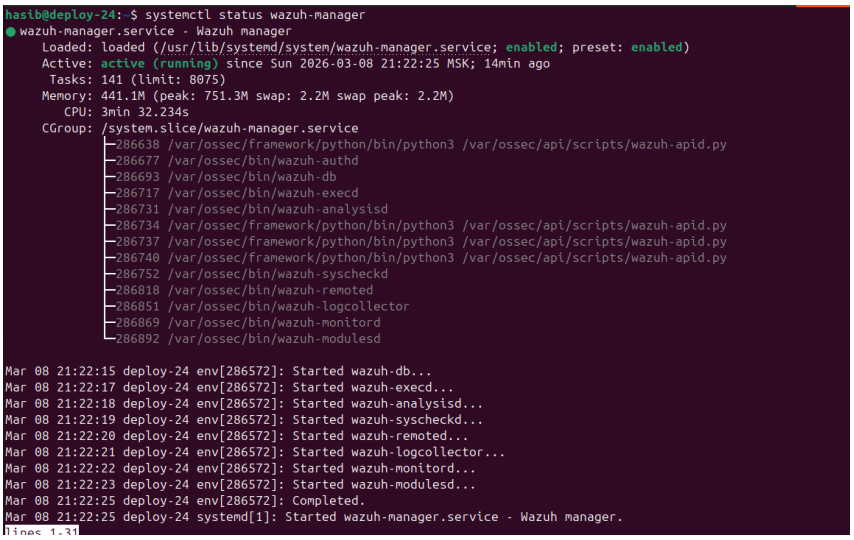
Service Status Check:

This is the database where the status is Active (running). Result Active (running) was found.

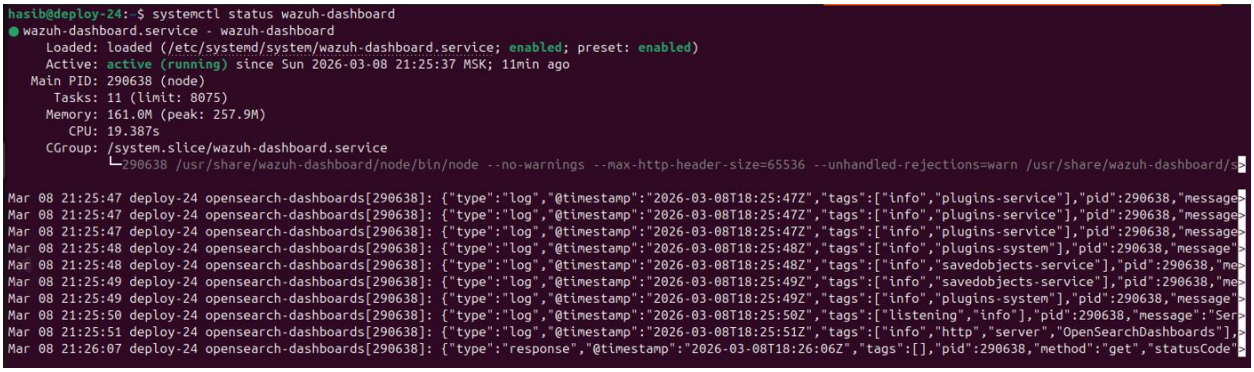
```
hasib@deploy-24:~$ sudo systemctl status wazuh-indexer
● wazuh-indexer.service - Wazuh-indexer
     Loaded: loaded (/usr/lib/systemd/system/wazuh-indexer.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-03-08 21:17:52 MSK; 18min ago
       Docs: https://documentation.wazuh.com
   Main PID: 242843 (java)
      Tasks: 93 (limit: 8075)
     Memory: 2.5G (peak: 3.7G swap: 1.2G swap peak: 1.2G)
        CPU: 2min 59.869s
     CGroup: /system.slice/wazuh-indexer.service
             └─242843 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch.networkaddress.cache.ttl=60 -Dopensearch.networkaddress.cache.negative.ttl=

Mar 08 21:17:13 deploy-24 systemd[1]: Starting wazuh-indexer.service - Wazuh-indexer...
Mar 08 21:17:30 deploy-24 systemd-entrypoint[242843]: WARNING: A terminally deprecated method in java.lang.System has been called
Mar 08 21:17:30 deploy-24 systemd-entrypoint[242843]: WARNING: System::setSecurityManager has been called by org.opensearch.bootstrap.OpenSearch (file:/usr/share/
Mar 08 21:17:30 deploy-24 systemd-entrypoint[242843]: WARNING: Please consider reporting this to the maintainers of org.opensearch.bootstrap.OpenSearch
Mar 08 21:17:30 deploy-24 systemd-entrypoint[242843]: WARNING: System::setSecurityManager will be removed in a future release
Mar 08 21:17:33 deploy-24 systemd-entrypoint[242843]: WARNING: A terminally deprecated method in java.lang.System has been called
Mar 08 21:17:33 deploy-24 systemd-entrypoint[242843]: WARNING: System::setSecurityManager has been called by org.opensearch.bootstrap.Security (file:/usr/share/wa
Mar 08 21:17:33 deploy-24 systemd-entrypoint[242843]: WARNING: Please consider reporting this to the maintainers of org.opensearch.bootstrap.Security
Mar 08 21:17:33 deploy-24 systemd-entrypoint[242843]: WARNING: System::setSecurityManager will be removed in a future release
Mar 08 21:17:52 deploy-24 systemd[1]: Started wazuh-indexer.service - Wazuh-indexer.
lines 1-21/21 (END)
```

This is the brain or processor that analyzes the log, where the status is found to be Active (running).



This is the web interface or UI. Here the status Active (running) was found by running the following command.



This means that it has been possible to successfully and stably deploy the Wazuh SIEM central node on the Manager server, eliminating all conflicts.

Multi-Device Configuration

The main goal of this task was to connect three different types of Heterogeneous devices to Wazuh SIEM and bring their log data to a central dashboard. I successfully connected Linux, Windows and a Network Device (Simulated).

Device 1: Linux

Device 1: Endpoint (VM-K8S)

OS: Ubuntu 22.04

Method: Wazuh Agent (Installed via .deb package)

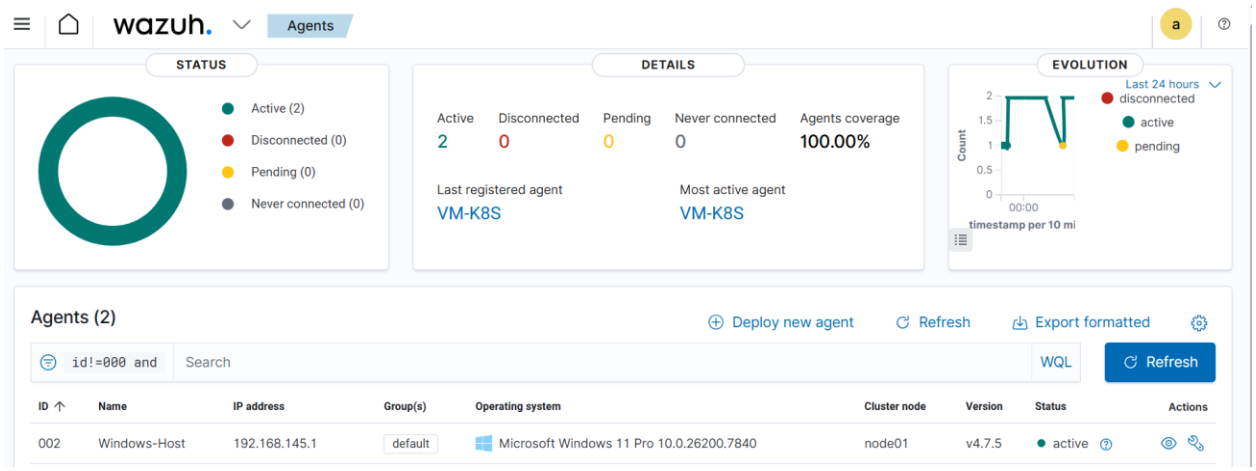
Configuration: The agent is registered with the manager's IP (192.168.145.146).

Status: It shows Successfully Active on the dashboard.



Device 2: Windows Endpoint (Windows-Host)

- **OS:** Windows Server/Desktop
- **Method:** MSI Installer (Wazuh Agent)
- **Status:** It shows Successfully Active on the dashboard.



Device 3: Network Device (Syslog-based)

- **Method:** Remote Syslog (UDP 514)
- **Configuration (on deploy-24):** Remote config has been added to the manager's ossec.conf file to retrieve data from network devices.

Network devices (such as routers or firewalls) usually do not support installing agents. Therefore, we have used the Syslog protocol.

```
<remote>
  <connection>secure</connection>
  <port>1514</port>
  <protocol>tcp</protocol>
  <queue_size>131072</queue_size>
</remote>

<remote>
  <connection>syslog</connection>
  <port>514</port>
  <protocol>udp</protocol>
  <allowed-ips>192.168.145.0/24</allowed-ips>
</remote>
```

Here the allowed-ips subnet is used so that any device on our lab network can send logs. After the configuration, the service is restarted by running the following command.

```
hasib@deploy-24:~$ sudo systemctl restart wazuh-manager
```

Checking the port: Checking whether port 514 is actually open or not by running the following command:

```
hasib@deploy-24:~$ sudo ss -ulpn | grep :514
UNCONN 0      0                  0.0.0.0:514        0.0.0.0:*    users:(("wazuh-remoted",pid=10408,fd=4))
```

The output shows UNCONN 0.0.0.0:514 and the wazuh-remoted process. This means that the manager is now ready to take logs from the network.

Log Simulation and Verification:

Here, the logs of a Cisco ASA firewall have been simulated and tested.

```
hasib@deploy-24:~$ logger -d -n 192.168.145.146 -P 514 "router-01 firewall: %ASA-4-106023: Deny tcp src outside:1.1.1.1 dst inside:10.0.0.5"
```

The logger command is used to send syslog messages to Linux. The -d flag confirms the UDP packet and -n indicates the IP of the manager. This simulates a "Deny" traffic alert.

Root Level Log Check (Archives Log): The archives.json file is monitored to see if the log is arriving properly inside Wazuh:

```
hasib@deploy-24:~$ sudo tail -f /var/ossec/logs/archives/archives.json | grep "router-01"
{"timestamp":"2026-03-11T17:46:35.552+0300","rule":{"level":3,"description":"ASA warning message.","id":"64004","firedtimes":1,"mail":false,"groups":["syslog","cis
co","cisco-asa"]},"agent":{"id":"000","name":"deploy-24"},"manager":{"name":"deploy-24"},"id":"1773240395.2281415","full_log":"1 2026-03-11T17:46:35.551063+03:00 d
eploy-24 hasib - - [timeQuality tzKnown=\"1\" isSynced=\"0\"] router-01 firewall: %ASA-4-106023: Deny tcp src outside:1.1.1.1 dst inside:10.0.0.5","decoder":{"pare
nt":"cisco-asa","name":"cisco-asa"},"location":"192.168.145.146"}
```

The output shows that the log was successfully detected. The output shows that Wazuh's default decoder (cisco-asa) recognized the log and triggered Rule ID: 64004 (ASA warning message).

Network Traffic Analysis:

The packet level was checked using tcpdump using the following command.

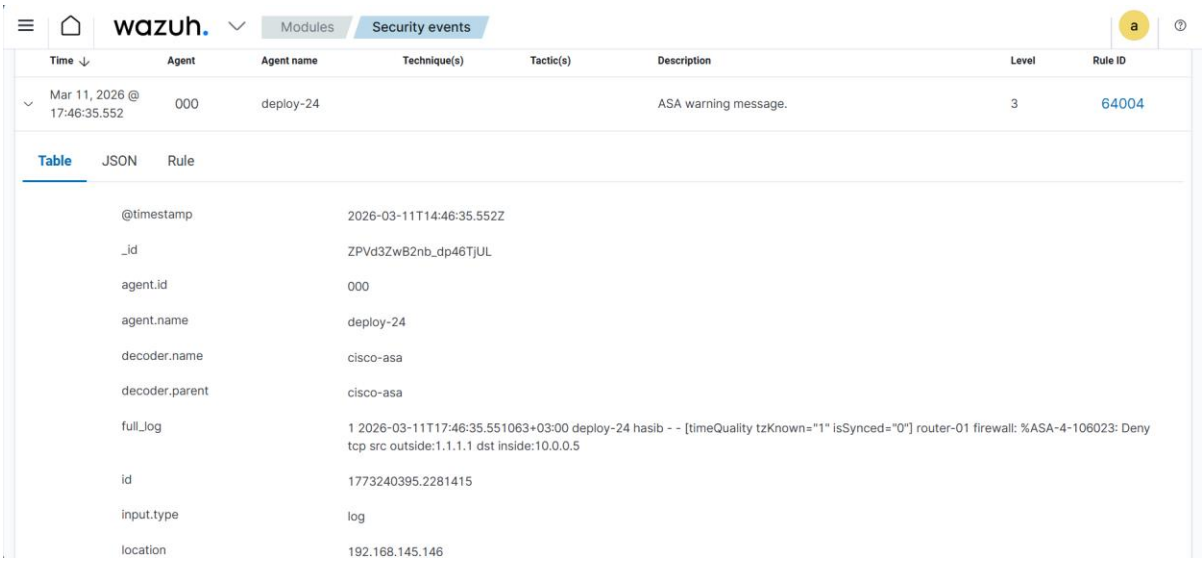
```
hasib@deploy-24:~$ sudo tcpdump -i any udp port 514 -A
[sudo] password for hasib:
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
```

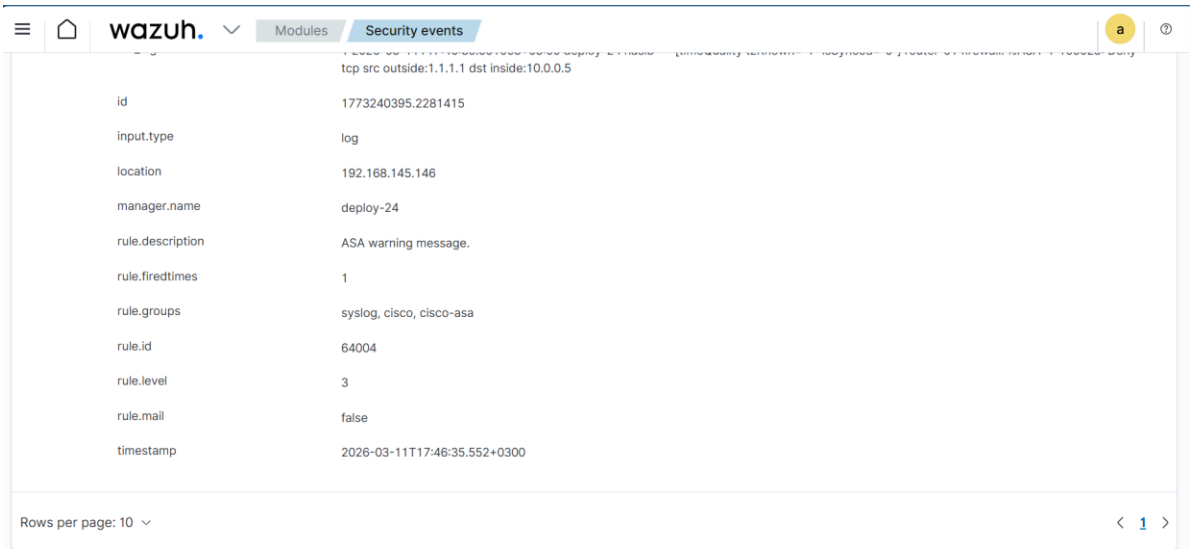
It shows every UDP 514 packet coming to the network interface in Raw format. This ensures that there is no firewall block at the network level.

Then the Manager is restarted again.

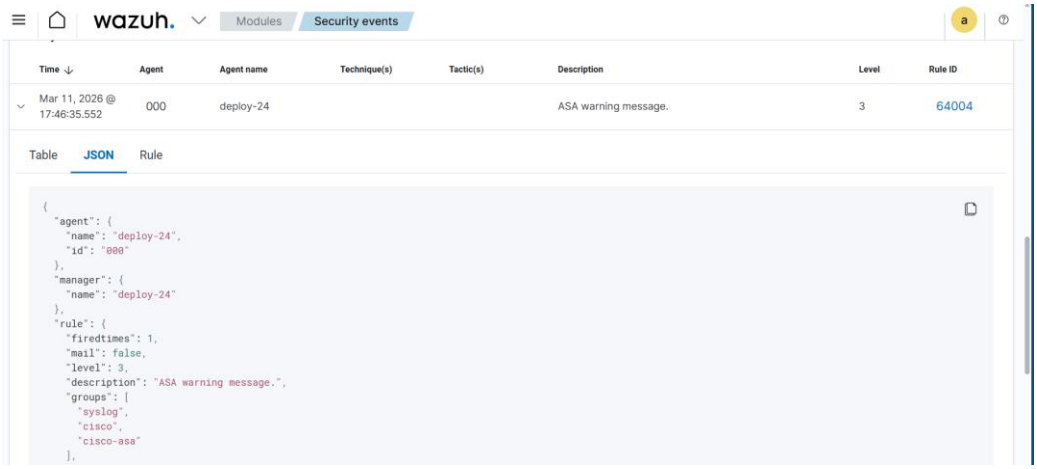
```
hasib@deploy-24:~$ sudo systemctl restart wazuh-manager
hasib@deploy-24:~$
```

Then a Security Alerts message was found on the Dashboard.





Log data from JSON file:



After the log arrived on the dashboard, the following information was found:

- **Rule Level:** 3 (Warning)
- **Description:** ASA warning message.
- **Full Log:** The specific error message that was sent by the Cisco router.

Proof of Manager Identity & Role:

The role of the Manager in a SIEM infrastructure is to act as the central authority. In our lab, the Manager machine is performing this role.

A. Identity and Network Verification:

The following command is useful to verify the hostname and IP of the Manager:

```
hasib@deploy-24:~$ hostname && hostname -I
deploy-24
10.10.10.13 192.168.145.146 172.18.0.1 192.168.49.1 172.17.0.1
```

In the above command, the hostname command shows the machine name (deploy-24) and hostname -I lists all the IP addresses associated with the machine. Here 192.168.145.146 is the primary IP of the manager, which is used for all agent communication.

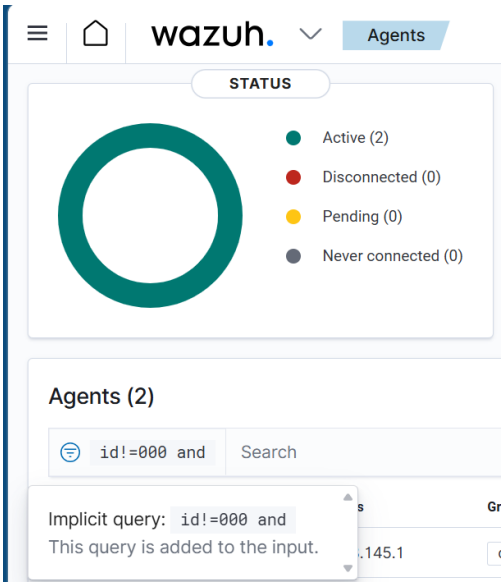
Service Status and Component Analysis:

The following command is used to verify whether the internal modules of Wazuh are working properly or not.

```
hasib@deploy-24:~$ sudo /var/ossec/bin/wazuh-control status
[sudo] password for hasib:
wazuh-clusterd not running...
wazuh-modulesd is running...
wazuh-monitord is running...
wazuh-logcollector is running...
wazuh-remoted is running...
wazuh-syscheckd is running...
wazuh-analysisd is running...
wazuh-maild not running...
wazuh-execd is running...
wazuh-db is running...
wazuh-authd is running...
wazuh-agentlessd not running...
wazuh-integratord is running...
wazuh-dbd not running...
wazuh-csyslogd not running...
wazuh-apid is running...
```

The above command shows the real-time status of each "sub-service" or module of the manager. wazuh-analysisd: This is the "brain" of the manager, which matches incoming logs with rules. wazuh-remoted is the ear that listens for messages from agents, and wazuh-syscheckd controls File Integrity Monitoring (FIM). The Running status here proves that the manager is fully capable of processing log data and sending mitigation commands.

After going to the Agents (Agent 000): section of the Wazuh dashboard, you will see that ID 000 is named deploy-24 and has a "Manager" tag next to it.



Device 4: Apache on Linux:

Apache access log monitoring on Linux endpoint:

Apache HTTP Server has been setup on VM-K8S endpoint to monitor a "Web Resource" as per the requirements of the lab manual.

Apache Installation and Service Status:

VM-k8S / 192.168.145.144

```
hasib@VM-K8S:~$ sudo apt update
[sudo] password for hasib:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://ru.archive.ubuntu.com/ubuntu noble InRelease
Hit:3 https://packages.wazuh.com/4.x/apt stable InRelease
Hit:4 https://download.docker.com/linux/ubuntu noble InRelease
Get:5 http://ru.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:7 https://packages.gitlab.com/runner/gitlab-runner/ubuntu noble InRelease
Get:8 http://ru.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,507 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.6 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.1 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [976 kB]
Get:14 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,807 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [218 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:17 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [20.6 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:19 http://ru.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [332 kB]
Get:20 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:21 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.7 kB]
Get:22 http://ru.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:23 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,565 kB]
Get:24 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:25 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [32.9 kB]
Get:26 http://ru.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:27 http://ru.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7,288 B]
Get:28 http://ru.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [216 B]
Get:29 http://ru.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.5 kB]
Get:30 http://ru.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
```

Installing Apache:

The following command installs the web server.

```
hasib@VM-K8S:~$ sudo apt install apache2 -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
apache2 is already the newest version (2.4.58-1ubuntu8.11).
The following packages were automatically installed and are no longer required:
  libcdio-cdda2t64 libcdio-paranoia2t64 libcdio19t64 libgdata-common libmsgraph-0-1 libmtp-common libmtp-runtime libmtp9t64 libnfs14
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 17 not upgraded.
```

Service status:

The enable command ensures that the server will start automatically even if the PC is restarted.

```
hasib@VM-K8S:~$ sudo systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
```

Starting apache2:

```
hasib@VM-K8S:~$ sudo systemctl start apache2
```

The Active (running) output of the command below confirms that the "Web Infrastructure" is now online.

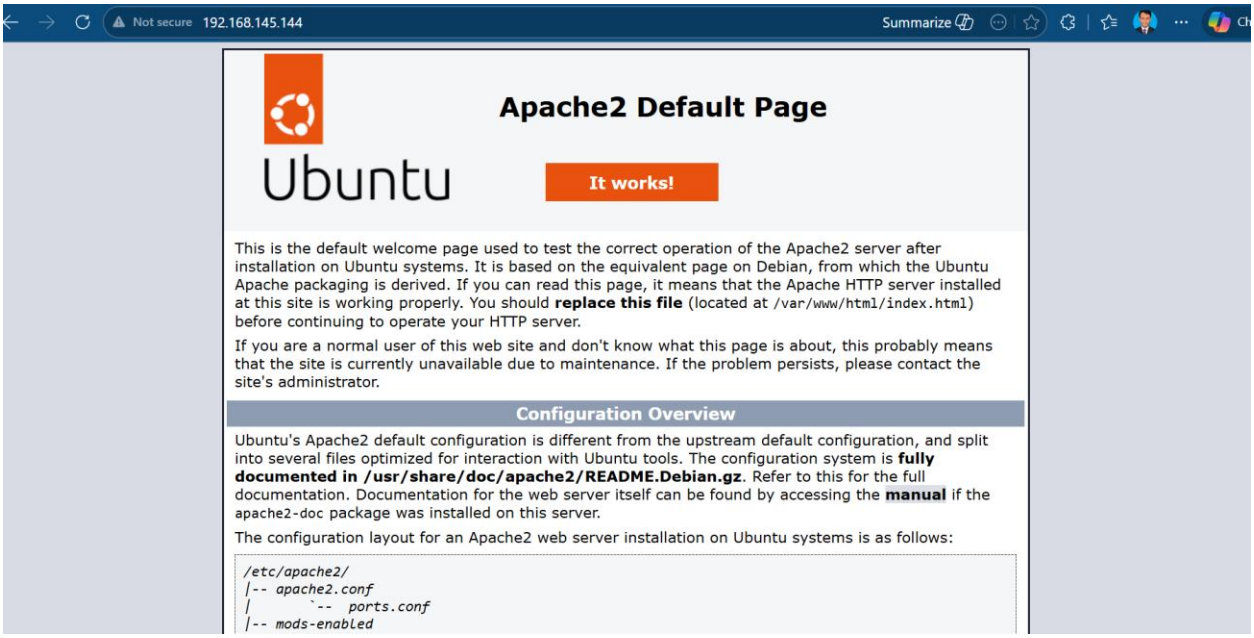
```
hasib@VM-K8S:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-03-11 13:03:32 MSK; 6h ago
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 1749 (apache2)
      Tasks: 55 (limit: 9373)
     Memory: 8.2M (peak: 8.9M)
        CPU: 2.503s
     CGroup: /system.slice/apache2.service
             ├─1749 /usr/sbin/apache2 -k start
             ├─1767 /usr/sbin/apache2 -k start
             └─1769 /usr/sbin/apache2 -k start

Mar 11 13:03:30 VM-K8S systemd[1]: Starting apache2.service - The Apache HTTP Server...
Mar 11 13:03:32 VM-K8S apachectl[1707]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerN
Mar 11 13:03:32 VM-K8S systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-16/16 (END)
```

Browser test:

From browser:

http://192.168.145.144



Access Log Analysis:

Whenever someone visits the server, Apache stores a record of it in the /var/log/apache2/access.log file.

```
hasib@VM-K8S:~$ sudo tail -f /var/log/apache2/access.log
[sudo] password for hasib:
192.168.145.1 - - [11/Mar/2026:00:55:11 +0300] "-" 408 0 "-" "-"
192.168.145.1 - - [11/Mar/2026:00:55:11 +0300] "-" 408 0 "-" "-"
192.168.145.1 - - [11/Mar/2026:16:44:48 +0300] "GET /favicon.ico HTTP/1.1" 404 494 "http://192.168.145.144/hasib-final-report" "Mozilla/5.0 (Windows NT 10.0; Win64
; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36 Edg/145.0.0.0"
192.168.145.1 - - [11/Mar/2026:16:45:40 +0300] "-" 408 0 "-" "-"
192.168.145.1 - - [11/Mar/2026:19:52:33 +0300] "GET / HTTP/1.1" 200 3460 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chro
me/145.0.0.0 Safari/537.36 Edg/145.0.0.0"
192.168.145.1 - - [11/Mar/2026:19:53:24 +0300] "-" 408 0 "-" "-"
^C
```

The output shows 200 (OK) and 404 (Not Found) requests coming from 192.168.145.1 (Host machine). This proves that the server is accepting the request and generating logs.

Log Collection Configuration via Wazuh Agent:

Just generating logs is not enough, SIEM needs to be able to read those logs. For this, I edited the agent configuration file and added the following config.

Added inside <ossec_config>:

```
 GNU nano 7.2                                                    /var/ossec/etc/

 <localfile>
   <log_format>syslog</log_format>
   <location>/var/log/auth.log</location>
 </localfile>

 <localfile>
   <log_format>syslog</log_format>
   <location>/var/log/syslog</location>
 </localfile>

 <localfile>
   <log_format>syslog</log_format>
   <location>/var/log/dpkg.log</location>
 </localfile>

 <localfile>
   <log_format>syslog</log_format>
   <location>/var/log/kern.log</location>
 </localfile>

 <localfile>
   <log_format>apache</log_format>
   <location>/var/log/apache2/error.log</location>
 </localfile>
```

Here <log_format>apache</log_format> tells Wazuh that the log format is Apache-like, so it should use the web-accesslog decoder. <location> indicates the exact path to the logs. After saving the file and running sudo systemctl restart wazuh-agent , the agent started sending these logs to the manager in real-time.

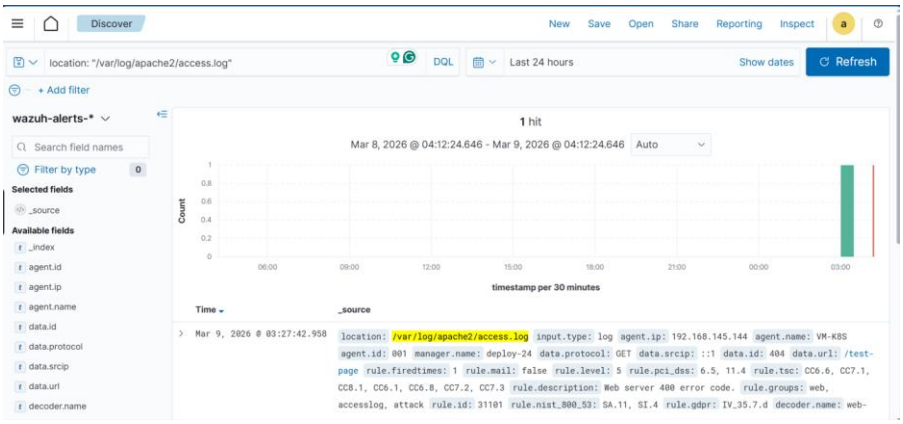
Restarting agent:

```
hasib@VM-K8S:~$ sudo systemctl restart wazuh-agent
```

Proof:

```
hasib@VM-K8S:~$ tail -f /var/log/apache2/access.log
192.168.145.1 - - [11/Mar/2026:00:55:11 +0300] "-" 408 0 "-" "-"
192.168.145.1 - - [11/Mar/2026:00:55:11 +0300] "-" 408 0 "-" "-"
192.168.145.1 - - [11/Mar/2026:16:44:48 +0300] "GET /favicon.ico HTTP/1.1" 404 494 "http://192.168.145.144/hasib-final-report" "Mozilla/5.0 (Windows NT 10.0; Win64
; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36 Edg/145.0.0.0"
192.168.145.1 - - [11/Mar/2026:16:45:40 +0300] "-" 408 0 "-" "-"
192.168.145.1 - - [11/Mar/2026:19:52:33 +0300] "GET / HTTP/1.1" 200 3460 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chro
me/145.0.0.0 Safari/537.36 Edg/145.0.0.0"
192.168.145.1 - - [11/Mar/2026:19:53:24 +0300] "-" 408 0 "-" "-"
```

Apache access events can be seen in the Dashboard by going to the Discover option.



How to view internal logs:

Internal logs are monitored to ensure the status of the SIEM Manager and whether its various modules (e.g. Vulnerability Detector, Analysis Engine) are working properly.

Real-time log monitoring from the terminal:

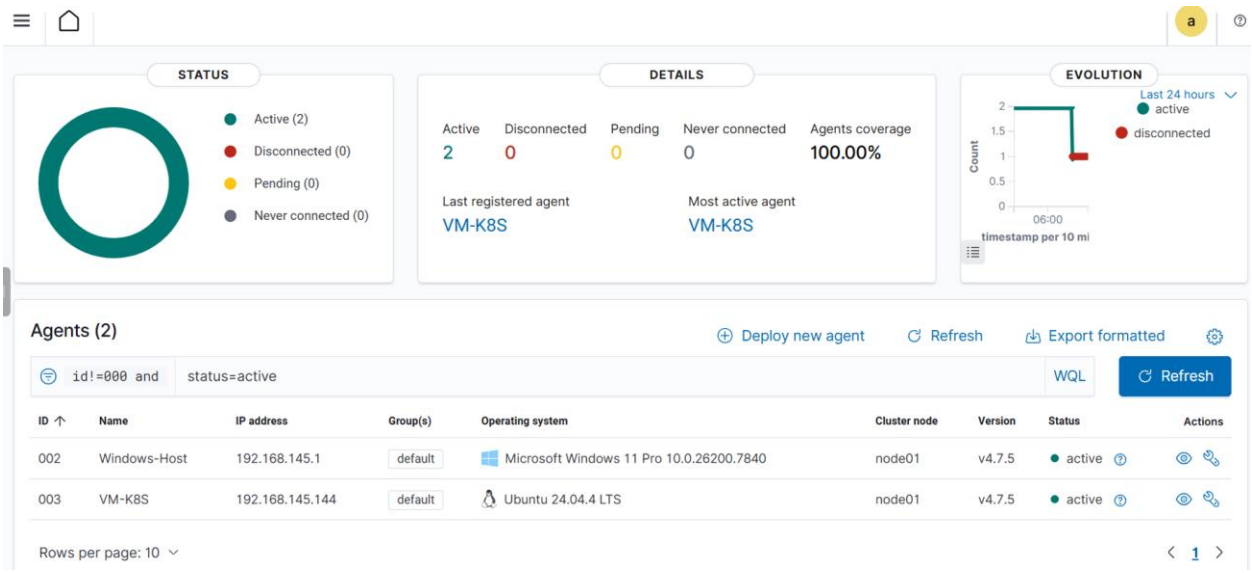
All back-end activities of the manager are stored in the /var/ossec/logs/ossec.log file.

```
hasib@deploy-24:~$ sudo tail -f /var/ossec/logs/ossec.log
2026/03/11 18:13:47 wazuh-modulesd:vulnerability-detector: INFO: (5471): Finished vulnerability assessment for agent '002'
2026/03/11 18:13:47 wazuh-modulesd:vulnerability-detector: INFO: (5450): Analyzing agent '003' vulnerabilities.
2026/03/11 18:13:47 wazuh-modulesd:vulnerability-detector: INFO: (5471): Finished vulnerability assessment for agent '003'
2026/03/11 18:13:47 wazuh-modulesd:vulnerability-detector: INFO: (5472): Vulnerability scan finished.
2026/03/11 18:18:47 wazuh-modulesd:vulnerability-detector: INFO: (5431): Starting vulnerability scan.
2026/03/11 18:18:47 wazuh-modulesd:vulnerability-detector: INFO: (5450): Analyzing agent '002' vulnerabilities.
2026/03/11 18:18:47 wazuh-modulesd:vulnerability-detector: INFO: (5471): Finished vulnerability assessment for agent '002'
2026/03/11 18:18:47 wazuh-modulesd:vulnerability-detector: INFO: (5450): Analyzing agent '003' vulnerabilities.
2026/03/11 18:18:47 wazuh-modulesd:vulnerability-detector: INFO: (5471): Finished vulnerability assessment for agent '003'
2026/03/11 18:18:47 wazuh-modulesd:vulnerability-detector: INFO: (5472): Vulnerability scan finished.
2026/03/11 18:23:47 wazuh-modulesd:vulnerability-detector: INFO: (5431): Starting vulnerability scan.
2026/03/11 18:23:48 wazuh-modulesd:vulnerability-detector: INFO: (5450): Analyzing agent '002' vulnerabilities.
2026/03/11 18:23:48 wazuh-modulesd:vulnerability-detector: INFO: (5471): Finished vulnerability assessment for agent '002'
2026/03/11 18:23:48 wazuh-modulesd:vulnerability-detector: INFO: (5450): Analyzing agent '003' vulnerabilities.
2026/03/11 18:23:48 wazuh-modulesd:vulnerability-detector: INFO: (5471): Finished vulnerability assessment for agent '003'
2026/03/11 18:23:48 wazuh-modulesd:vulnerability-detector: INFO: (5472): Vulnerability scan finished.
```

Here the tail command shows the end of a file. The -f (Follow) flag keeps the file open in real-time, meaning that as soon as a new log is generated in the system, it is displayed on the screen. The log shows that the wazuh-modulesd:vulnerability-detector module is active. It has completed Vulnerability Assessment for Agents 002 (Windows) and 003 (Linux). This means that the SIEM system is automatically detecting vulnerabilities on the endpoints.

Viewing Management Information from Dashboard (UI Management Method):

Then from the dashboard, go to Management and go to Status option to view Management Information. The text 'Server' confirms that the deploy-24 machine is working as a full-fledged SIEM Manager, not a simple agent. 2 active agents were seen in the dashboard (Windows-Host and VM-K8S). This proves that the agent-manager connectivity is successful.



Network Scalability:

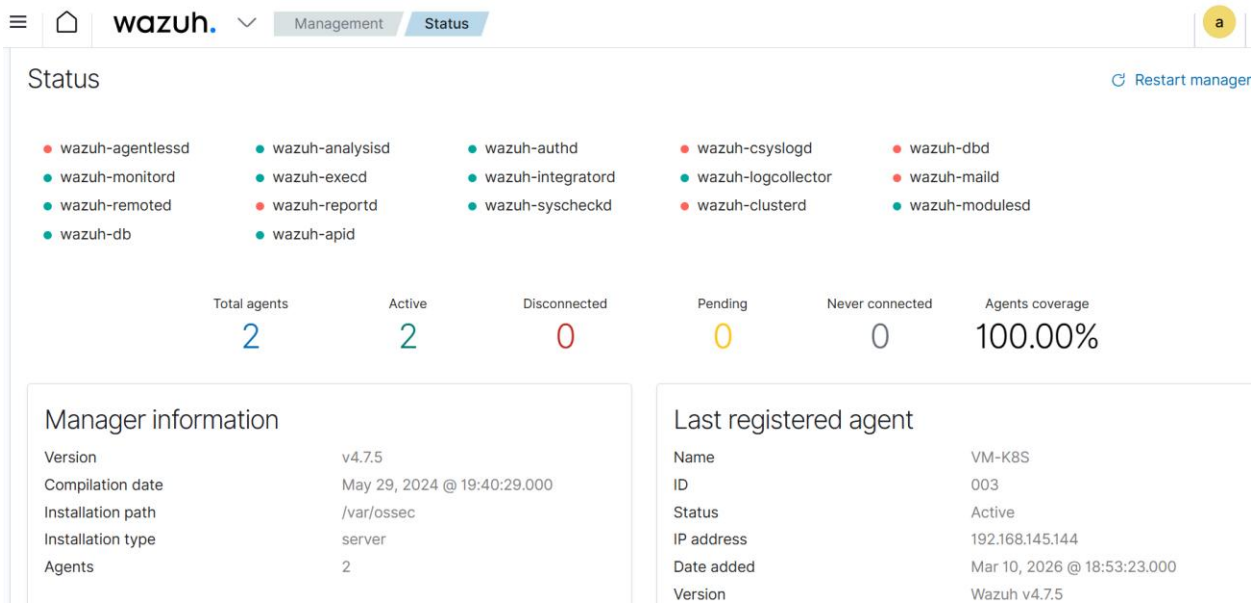
Here Agents: 2 are shown. This proves the successful connectivity of the project:

• **Agent 1:** Windows machine (Windows-Host).

• **Agent 2:** Linux node (VM-K8S). (Since device number 3 is connected via syslog, it is not shown here in the number but is shown as a source in the 'Management' tab.)

Version and Path (Standardization):

Version v4.7.5: Latest and stable version used. Path /var/ossec Standard Linux directory, which demonstrates Linux administration skills.

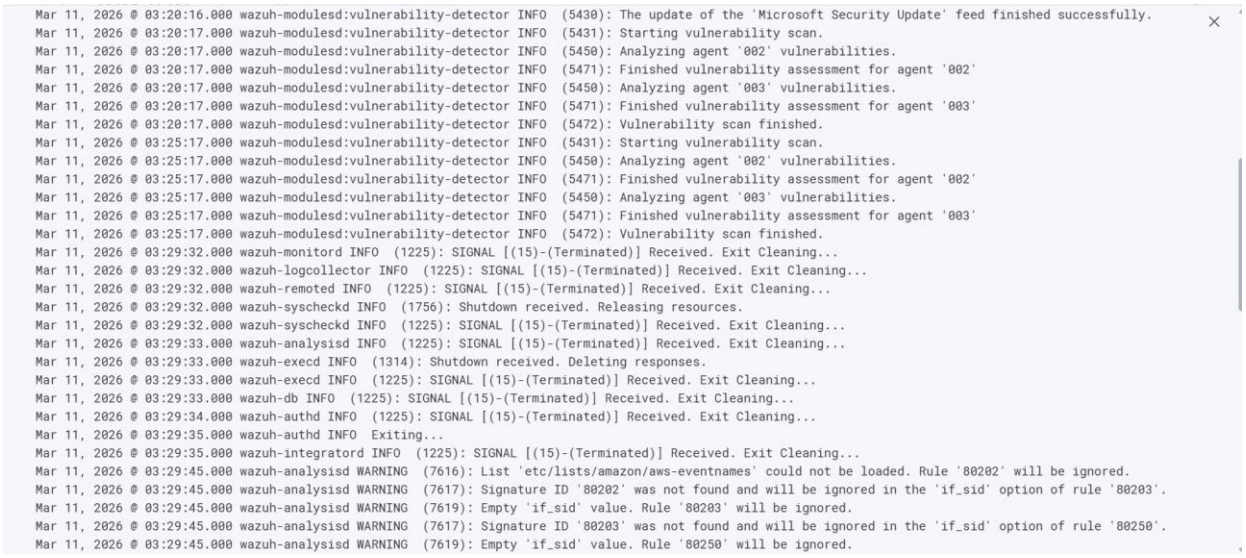


Dashboard UI Logs Analysis:

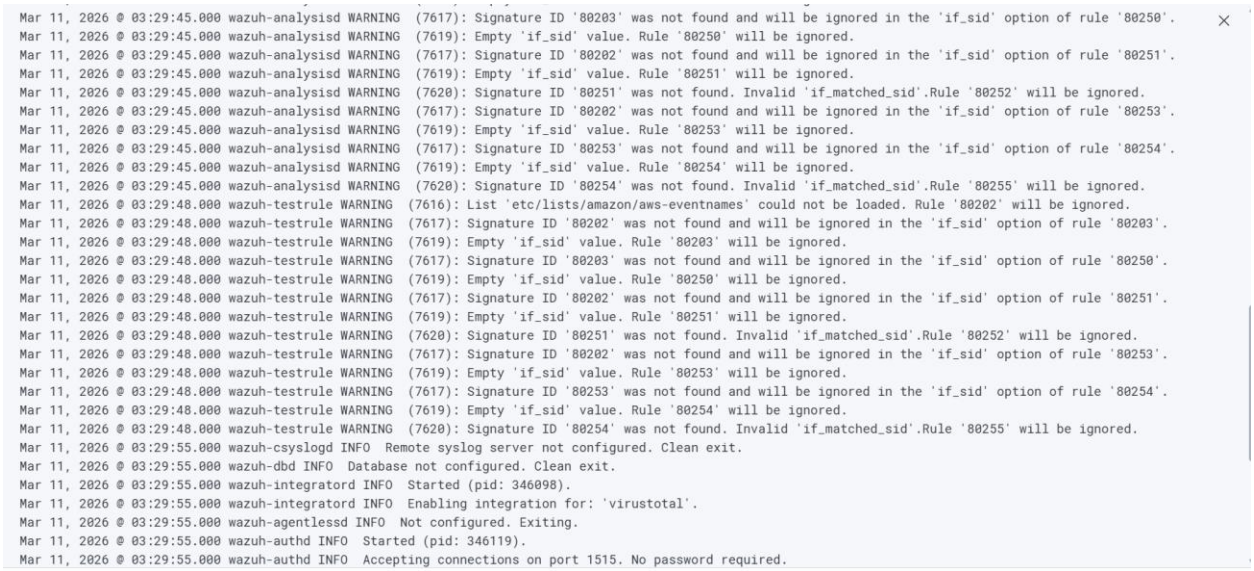
In addition to terminal logs, the manager's internal logs were analyzed using the dashboard's graphical interface. The manager's complete system logs were monitored by going to Management > Logs or Settings > Logs from the left-hand menu of the dashboard.

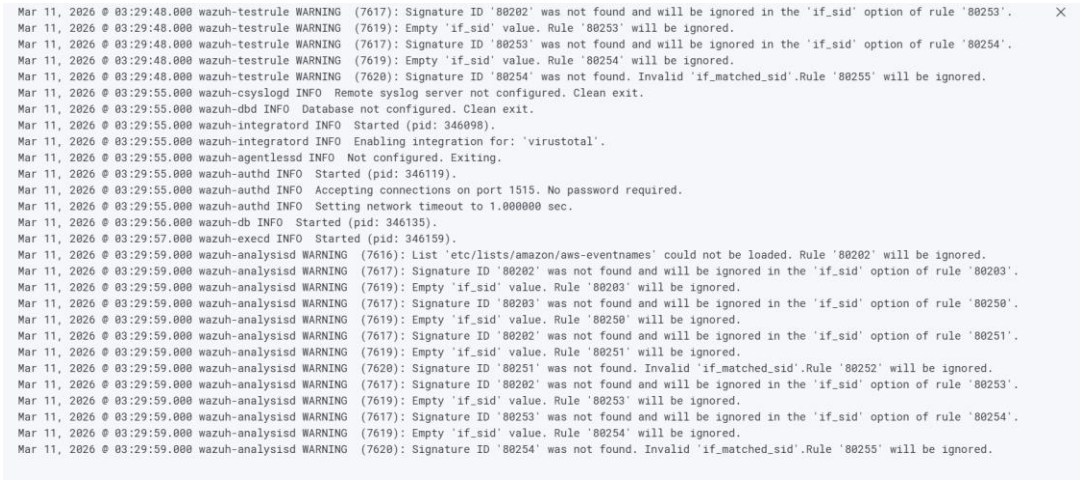


```
Mar 11, 2026 @ 03:00:10.000 wazuh-monitord INFO  Starting new log after rotation.
Mar 11, 2026 @ 03:03:20.000 wazuh-modulesd:vulnerability-detector INFO  (5431): Starting vulnerability scan.
Mar 11, 2026 @ 03:03:20.000 wazuh-modulesd:vulnerability-detector INFO  (5450): Analyzing agent '002' vulnerabilities.
Mar 11, 2026 @ 03:03:20.000 wazuh-modulesd:vulnerability-detector INFO  (5471): Finished vulnerability assessment for agent '002'
Mar 11, 2026 @ 03:03:20.000 wazuh-modulesd:vulnerability-detector INFO  (5450): Analyzing agent '003' vulnerabilities.
Mar 11, 2026 @ 03:03:20.000 wazuh-modulesd:vulnerability-detector INFO  (5471): Finished vulnerability assessment for agent '003'
Mar 11, 2026 @ 03:03:20.000 wazuh-modulesd:vulnerability-detector INFO  (5472): Vulnerability scan finished.
Mar 11, 2026 @ 03:08:20.000 wazuh-modulesd:vulnerability-detector INFO  (5431): Starting vulnerability scan.
Mar 11, 2026 @ 03:08:20.000 wazuh-modulesd:vulnerability-detector INFO  (5450): Analyzing agent '002' vulnerabilities.
Mar 11, 2026 @ 03:08:20.000 wazuh-modulesd:vulnerability-detector INFO  (5471): Finished vulnerability assessment for agent '002'
Mar 11, 2026 @ 03:08:20.000 wazuh-modulesd:vulnerability-detector INFO  (5450): Analyzing agent '003' vulnerabilities.
Mar 11, 2026 @ 03:08:20.000 wazuh-modulesd:vulnerability-detector INFO  (5471): Finished vulnerability assessment for agent '003'
Mar 11, 2026 @ 03:08:20.000 wazuh-modulesd:vulnerability-detector INFO  (5472): Vulnerability scan finished.
Mar 11, 2026 @ 03:11:48.000 wazuh-modulesd:vulnerability-detector INFO  (5400): Starting 'Ubuntu Trusty' database update.
Mar 11, 2026 @ 03:13:07.000 wazuh-modulesd:vulnerability-detector INFO  (5430): The update of the 'Ubuntu Trusty' feed finished successfully.
Mar 11, 2026 @ 03:13:07.000 wazuh-modulesd:vulnerability-detector INFO  (5400): Starting 'Ubuntu Xenial' database update.
Mar 11, 2026 @ 03:14:14.000 wazuh-modulesd:vulnerability-detector INFO  (5430): The update of the 'Ubuntu Xenial' feed finished successfully.
Mar 11, 2026 @ 03:14:14.000 wazuh-modulesd:vulnerability-detector INFO  (5400): Starting 'Ubuntu Bionic' database update.
Mar 11, 2026 @ 03:15:38.000 wazuh-modulesd:vulnerability-detector INFO  (5430): The update of the 'Ubuntu Bionic' feed finished successfully.
Mar 11, 2026 @ 03:15:38.000 wazuh-modulesd:vulnerability-detector INFO  (5400): Starting 'Ubuntu Focal' database update.
Mar 11, 2026 @ 03:17:39.000 wazuh-modulesd:vulnerability-detector INFO  (5430): The update of the 'Ubuntu Focal' feed finished successfully.
Mar 11, 2026 @ 03:17:39.000 wazuh-modulesd:vulnerability-detector INFO  (5400): Starting 'Ubuntu Jammy' database update.
Mar 11, 2026 @ 03:20:11.000 wazuh-modulesd:vulnerability-detector INFO  (5430): The update of the 'Ubuntu Jammy' feed finished successfully.
Mar 11, 2026 @ 03:20:11.000 wazuh-modulesd:vulnerability-detector INFO  (5400): Starting 'National Vulnerability Database' database update.
Mar 11, 2026 @ 03:20:15.000 wazuh-modulesd:vulnerability-detector INFO  (5430): The update of the 'National Vulnerability Database' feed finished successfully.
Mar 11, 2026 @ 03:20:15.000 wazuh-modulesd:vulnerability-detector INFO  (5400): Starting 'Microsoft Security Update' database update.
Mar 11, 2026 @ 03:20:16.000 wazuh-modulesd:vulnerability-detector INFO  (5430): The update of the 'Microsoft Security Update' feed finished successfully.
Mar 11, 2026 @ 03:20:17.000 wazuh-modulesd:vulnerability-detector INFO  (5431): Starting vulnerability scan.
```



```
Mar 11, 2026 @ 03:20:16.000 wazuh-modulesd:vulnerability-detector INFO  (5430): The update of the 'Microsoft Security Update' feed finished successfully.
Mar 11, 2026 @ 03:20:17.000 wazuh-modulesd:vulnerability-detector INFO  (5431): Starting vulnerability scan.
Mar 11, 2026 @ 03:20:17.000 wazuh-modulesd:vulnerability-detector INFO  (5450): Analyzing agent '002' vulnerabilities.
Mar 11, 2026 @ 03:20:17.000 wazuh-modulesd:vulnerability-detector INFO  (5471): Finished vulnerability assessment for agent '002'
Mar 11, 2026 @ 03:20:17.000 wazuh-modulesd:vulnerability-detector INFO  (5450): Analyzing agent '003' vulnerabilities.
Mar 11, 2026 @ 03:20:17.000 wazuh-modulesd:vulnerability-detector INFO  (5471): Finished vulnerability assessment for agent '003'
Mar 11, 2026 @ 03:20:17.000 wazuh-modulesd:vulnerability-detector INFO  (5472): Vulnerability scan finished.
Mar 11, 2026 @ 03:25:17.000 wazuh-modulesd:vulnerability-detector INFO  (5431): Starting vulnerability scan.
Mar 11, 2026 @ 03:25:17.000 wazuh-modulesd:vulnerability-detector INFO  (5450): Analyzing agent '002' vulnerabilities.
Mar 11, 2026 @ 03:25:17.000 wazuh-modulesd:vulnerability-detector INFO  (5471): Finished vulnerability assessment for agent '002'
Mar 11, 2026 @ 03:25:17.000 wazuh-modulesd:vulnerability-detector INFO  (5450): Analyzing agent '003' vulnerabilities.
Mar 11, 2026 @ 03:25:17.000 wazuh-modulesd:vulnerability-detector INFO  (5471): Finished vulnerability assessment for agent '003'
Mar 11, 2026 @ 03:25:17.000 wazuh-modulesd:vulnerability-detector INFO  (5472): Vulnerability scan finished.
Mar 11, 2026 @ 03:29:32.000 wazuh-monitord INFO  (1225): SIGNAL [(15)-(Terminated)] Received. Exit Cleaning...
Mar 11, 2026 @ 03:29:32.000 wazuh-logcollector INFO  (1225): SIGNAL [(15)-(Terminated)] Received. Exit Cleaning...
Mar 11, 2026 @ 03:29:32.000 wazuh-remoted INFO  (1225): SIGNAL [(15)-(Terminated)] Received. Exit Cleaning...
Mar 11, 2026 @ 03:29:32.000 wazuh-syscheckd INFO  (1756): Shutdown received. Releasing resources.
Mar 11, 2026 @ 03:29:32.000 wazuh-syscheckd INFO  (1225): SIGNAL [(15)-(Terminated)] Received. Exit Cleaning...
Mar 11, 2026 @ 03:29:33.000 wazuh-analysisd INFO  (1225): SIGNAL [(15)-(Terminated)] Received. Exit Cleaning...
Mar 11, 2026 @ 03:29:33.000 wazuh-execd INFO  (1314): Shutdown received. Deleting responses.
Mar 11, 2026 @ 03:29:33.000 wazuh-execd INFO  (1225): SIGNAL [(15)-(Terminated)] Received. Exit Cleaning...
Mar 11, 2026 @ 03:29:33.000 wazuh-db INFO  (1225): SIGNAL [(15)-(Terminated)] Received. Exit Cleaning...
Mar 11, 2026 @ 03:29:34.000 wazuh-authd INFO  (1225): SIGNAL [(15)-(Terminated)] Received. Exit Cleaning...
Mar 11, 2026 @ 03:29:35.000 wazuh-authd INFO  Exiting...
Mar 11, 2026 @ 03:29:35.000 wazuh-integratord INFO  (1225): SIGNAL [(15)-(Terminated)] Received. Exit Cleaning...
Mar 11, 2026 @ 03:29:45.000 wazuh-analysisd WARNING  (7616): List 'etc/lists/amazon/aws-eventnames' could not be loaded. Rule '80202' will be ignored.
Mar 11, 2026 @ 03:29:45.000 wazuh-analysisd WARNING  (7617): Signature ID '80202' was not found and will be ignored in the 'if_sid' option of rule '80203'.
Mar 11, 2026 @ 03:29:45.000 wazuh-analysisd WARNING  (7619): Empty 'if_sid' value. Rule '80203' will be ignored.
Mar 11, 2026 @ 03:29:45.000 wazuh-analysisd WARNING  (7617): Signature ID '80203' was not found and will be ignored in the 'if_sid' option of rule '80250'.
Mar 11, 2026 @ 03:29:45.000 wazuh-analysisd WARNING  (7619): Empty 'if_sid' value. Rule '80250' will be ignored.
```





The important system activities from the collected logs are explained below:

1. Vulnerability Detection:

At the beginning of the log, the manager is regularly scanning two agents 002 (Windows) and 003 (Linux). Here, the manager is checking whether there are any security loopholes or outdated software on these two devices. As a result, the log says Finished vulnerability assessment for agent '002' and '003'. This means that the security audit of all devices has been successfully completed.

2. Database Updates:

The manager is updating its knowledge base. It has downloaded the latest virus information from Ubuntu (Trusty, Focal, Jammy), Microsoft Security, and NVD (National Vulnerability Database). This proves that your SIEM is able to recognize the latest threats.

3. Manager Restart Sequence:

In the middle of the log (at 03:29:32), it is seen that all services have received SIGNAL [(15)-(Terminated)]. This probably means that the manager restarted or the configuration was changed. After that, all modules (such as authd, db, execd) started successfully again.

4. VirusTotal Integration:

The best news for you is: wazuh-integratord INFO: Enabling integration for: 'virustotal' - This line is the mainstay of Malware removal. From now on, whenever the manager finds a malware file, it will automatically check it with VirusTotal.

5. AWS Warnings:

Some WARNINGs are appearing in the log (such as Rule 80202). This basically means that some AWS (Amazon Web Services) lists are not found. Since cloud monitoring was not done, these warnings will not cause any problems in your project.

b. Why specifically are you able to view these logs i.e select two visible logs, explain these logs, and explain why and how you are able to view it on the SIEM.

Solution:

This section shows how a specific event occurs on the endpoint and how it is visible as logs and alerts in Wazuh SIEM.

Log Selection 1: Linux Sudo/SSH Alerts (Rule ID: 5402)

Description: This alert is generated when a normal user on a VM-K8S (Linux) machine uses the sudo command to gain root privilege or admin power.

Practical Demonstration: In the lab, the following commands were run with root access on a VM-K8S machine to trigger this alert:

To become the root user:

```
hasib@VM-K8S:~$ sudo -i
[sudo] password for hasib:
```

To check the root directory:

```
root@VM-K8S:~# sudo ls -l /root/eicar_test.com
-rw-r--r-- 1 root root 69 Mar 11 03:41 /root/eicar_test.com
```

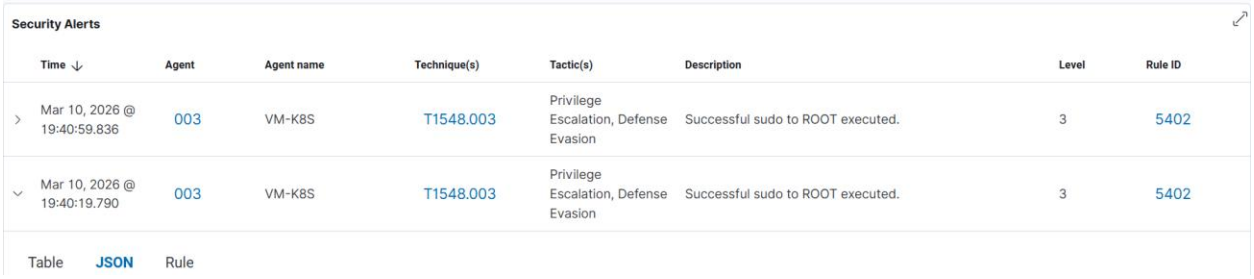
To check the Active Response Log:

```
root@VM-K8S:~# sudo tail -n 10 /var/ossec/logs/active-responses.log
Tue Mar 10 06:53:52 PM MSK 2026 active-response/bin/restart.sh agent
Wed Mar 11 12:30:03 AM MSK 2026 active-response/bin/restart.sh agent
Wed Mar 11 01:13:52 AM MSK 2026 [Active Response] Removed malware file: /root/ei
car.com
Wed Mar 11 01:13:59 AM MSK 2026 [Active Response] Removed malware file: /root/ei
car.com
Wed Mar 11 03:04:43 AM MSK 2026 active-response/bin/restart.sh agent
Wed Mar 11 03:50:12 AM MSK 2026 active-response/bin/restart.sh agent
```

You took root power from the user using sudo, so Linux's auth.log recorded it and Wazuh sent it to the manager.

Log Explanation:

Rule ID: 5402 issue found on Agent 003 in Dashboard, where Tactics (**Privilege Escalation, Defense Evasion**) was found with Description (**Successful sudo to Root executed**). The log was successfully mapped to the Privilege Escalation (**T1548.003**) tactic of the MITRE Framework for Cybersecurity. This data comes from the /var/log/auth.log file on the Linux endpoint.

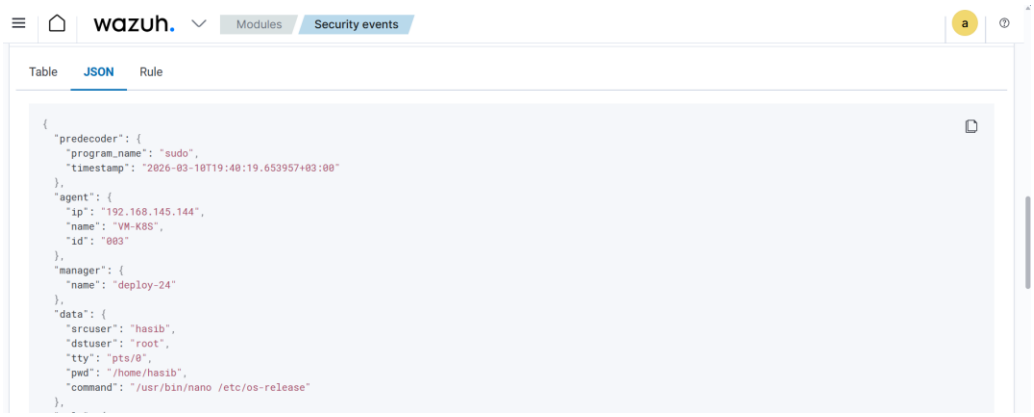


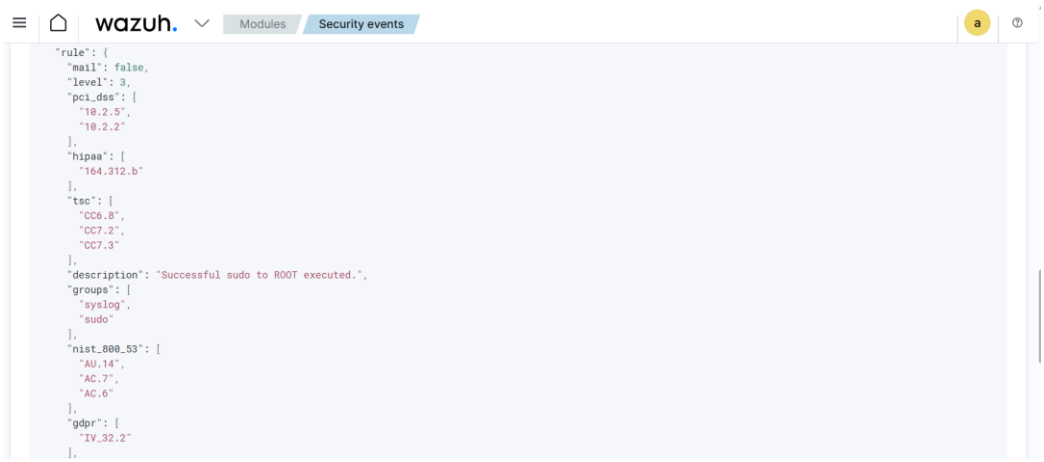
Security Alerts

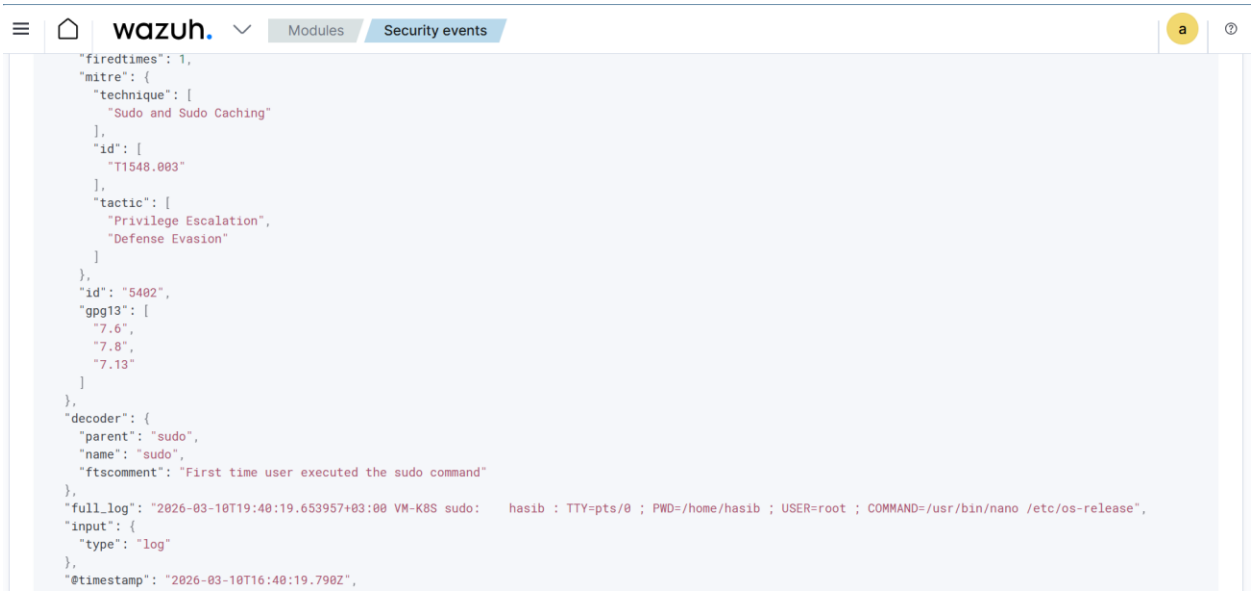
Time	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
Mar 10, 2026 @ 19:40:59.836	003	VM-K8S	T1548.003	Privilege Escalation, Defense Evasion	Successful sudo to ROOT executed.	3	5402
Mar 10, 2026 @ 19:40:19.790	003	VM-K8S	T1548.003	Privilege Escalation, Defense Evasion	Successful sudo to ROOT executed.	3	5402

Table JSON Rule

Log (JSON):





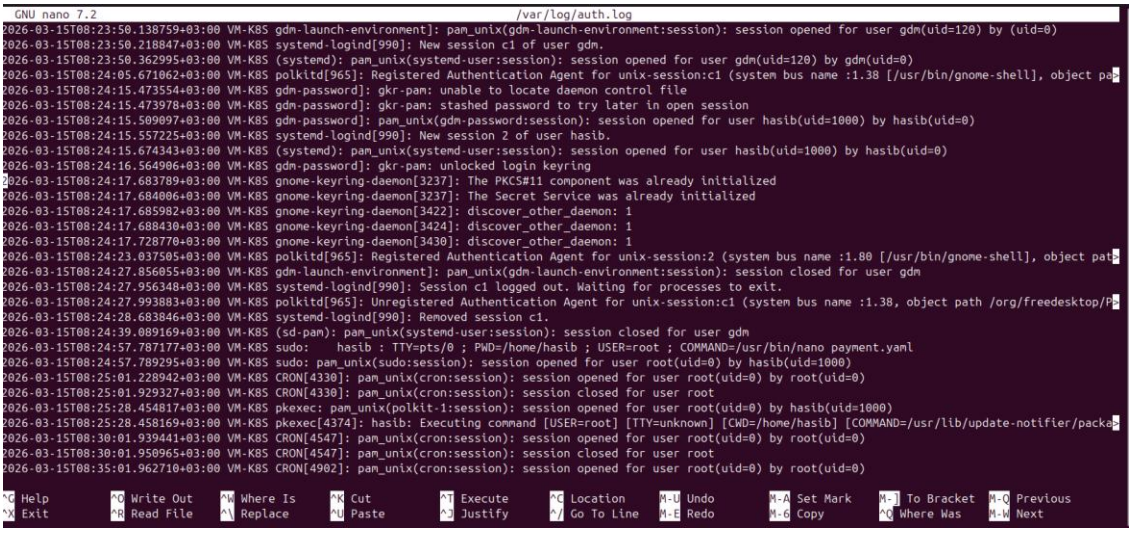


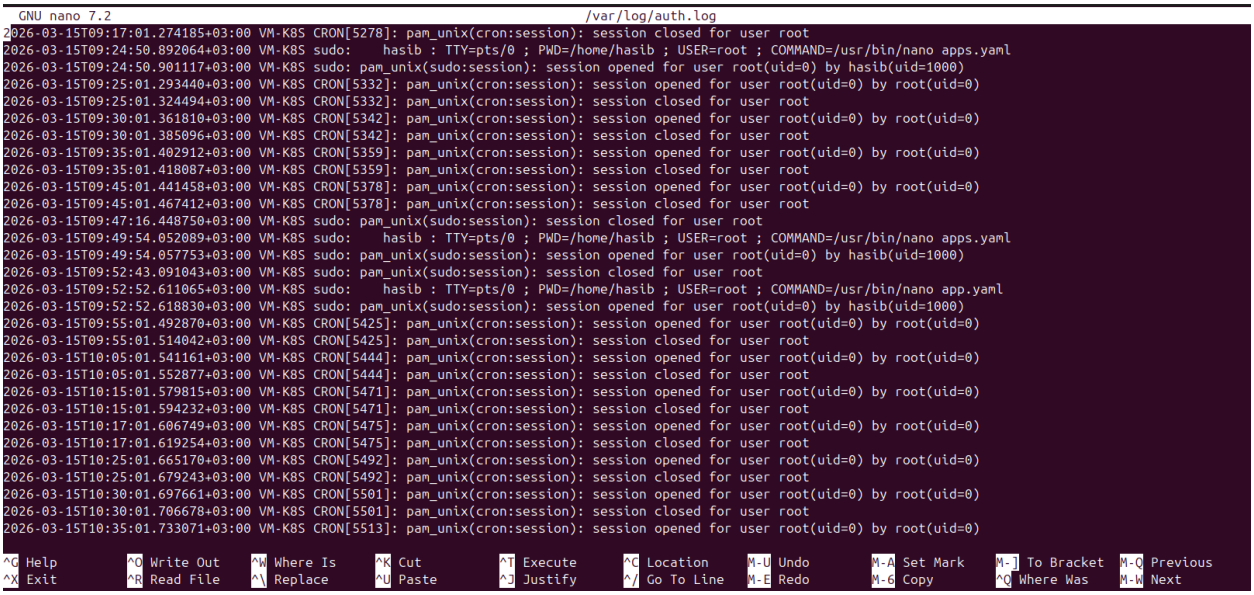

```
  "location": "/var/log/auth.log",
  "id": "1773160819.3966528",
  "timestamp": "2026-03-10T19:40:19.790+0300",
  "_id": "xeuf2JwBhTX6GBsmEPw3"
}
```


Reasons why logs are visible in SIEM (The "How" & "Why"):

The mechanism behind how a common Linux command appears as a security alert in the SIEM dashboard is given below:

How: The Wazuh agent is configured to read the default security log file /var/log/auth.log of the Linux system.





Why:

- **Continuous Monitoring:** Wazuh's Logcollector on VM-K8S (Agent 003) machine Wazuh's Logcollector module monitors the file in real-time and sends any new entries to the manager on an encrypted port, which is the job of the Logcollector.
- **Secure Transmission:** Whenever sudo is used, Linux writes that log to a file. The agent immediately encrypts that new log line and sends it to the specified port of the manager (Agent 000).
- **Decoding & Alerting:** The manager first reads the log using its Decoders and then sends it to the Rule Engine. The Decoder which was in this part of the Manger.

```
<syscheck>
  <disabled>no</disabled>
  <frequency>43200</frequency>
  <scan_on_start>yes</scan_on_start>
  <alert_new_files>yes</alert_new_files>
  <directories realtime="yes">/root</directories>
  <directories realtime="yes">/home/hasib</directories>
</syscheck>
```

When this log pattern matches **Rule 5402** (**Successful sudo to ROOT executed**), the manager saves it as a security alert on the dashboard and in alerts.json.

The "How":

This process of sending logs from the agent to the manager is made possible by our manual configuration. We have instructed the Linux Agent 003 to monitor a specific log file.

We have added the required block by opening the main configuration file of the agent with the following command:

```
hasib@VM-K8S:~$ sudo nano /var/ossec/etc/ossec.conf
```

```
  GNU nano 7.2                  /var/ossec/etc/ossec.conf
    <log_format>syslog</log_format>
    <location>/var/ossec/logs/active-responses.log</location>
  </localfile>

  <localfile>
    <log_format>syslog</log_format>
    <location>/var/log/auth.log</location>
  </localfile>
```

This configuration gives the agent 3 main instructions:

- **<location>:** The agent is told to log all authentication and privilege events on the Linux system to the /var/log/auth.log file. It should target this file.
- **<log_format>:** Since Linux writes these logs in syslog format by default, syslog is specified as the format so that the agent can easily parse the data.
- **Real-time Monitoring:** After saving this configuration, the agent starts reading the file continuously (Continuous monitoring). Whenever a new line (such as a record of the sudo command) is added to the file, the agent immediately reports it to the manager over an encrypted channel.

Log Selection 2: File Integrity Monitoring - FIM (Rule ID: 554)

Selected Log: File added to the system (Rule ID: 554)

Mar 11, 2026 @ 01:13:45.204	003	VM-K8S	File added to the system.	5	554

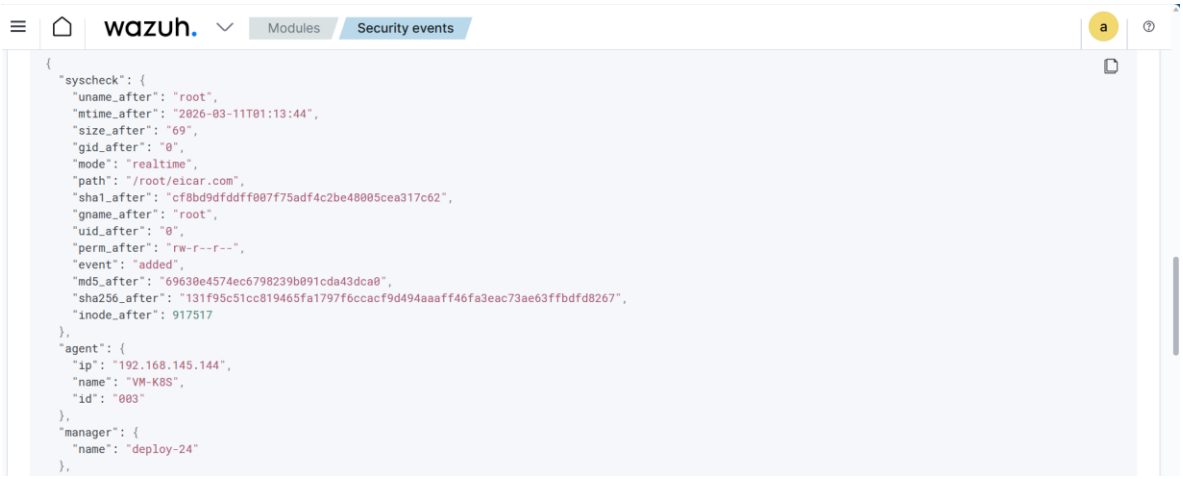
In this task, we tested how Wazuh can detect new files added to the system in real-time. This test was performed using a test malware file (EICAR).

Practical Demonstration (Executed Commands):

A standard antivirus test file was created using root privileges on the Agent 003 machine.

```
hasib@VM-K8S:~$ sudo -i
root@VM-K8S:~# echo 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' > /root/eicar.com
root@VM-K8S:~#
```

Log:




```
{
  "syscheck": {
    "uname_after": "root",
    "mtime_after": "2026-03-11T01:13:44",
    "size_after": "69",
    "gid_after": "0",
    "mode": "realtime",
    "path": "/root/eicar.com",
    "sha1_after": "cf8bd9dfddff007f75adf4c2be48005cea317c62",
    "gname_after": "root",
    "uid_after": "0",
    "perm_after": "rw-r--r--",
    "event": "added",
    "md5_after": "69630e4574ec6798239b091cda43dca0",
    "sha256_after": "131f95c51cc819465fa1797f6ccacf9d494aaaff46fa3eac73ae63ffbdfd8267",
    "inode_after": 917517
  },
  "agent": {
    "ip": "192.168.145.144",
    "name": "VM-K8S",
    "id": "003"
  },
  "manager": {
    "name": "deploy-24"
  },
  "rule": {
    "firedtimes": 1,
    "mail": false,
    "level": 5,
    "pci_dss": [
      "11.5"
    ],
    "hipaa": [
      "164.312.c.1",
      "164.312.c.2"
    ],
    "tsc": [
      "PI1.4",
      "PI1.5",
      "CC6.1",
      "CC6.8",
      "CC7.2",
      "CC7.3"
    ],
    "description": "File added to the system.",
    "groups": [
      "ossec",
      "syscheck",
      "syscheck_entry_added",
      "syscheck_file"
    ],
    "id": "554",
    "nist_800_53": [
      "SI.7"
    ],
    "gpg13": [
      "4.11"
    ],
    "gdpr": [
      "II_5.1.f"
    ]
  },
  "decoder": {
    "name": "syscheck_new_entry"
  },
  "full_log": "File '/root/eicar.com' added\nMode: realtime\n",
  "input": {
    "type": "log"
  },
  "@timestamp": "2026-03-10T22:13:45.204Z",
  "location": "syscheck",
  "id": "1773180825.292945",
  "timestamp": "2026-03-11T01:13:45.204+0300",
  "_id": "6ezQ2ZwBhTX6GBsmRwTU"
}
```

Log Analysis - JSON:

As soon as the file was created, **Rule ID: 554** was triggered in the Wazuh dashboard, clearly proving that the specific EICAR test file was added. The key parts of the received log are analyzed below:

• **Detected Path:** /root/eicar.com (The system correctly detected that a new file was added to the /root directory).

• **Monitoring Mode:** realtime This means that the file was detected by SIEM within 1 second of its creation.

• **File Hashes:** The md5 and sha256 hash values of the file are stored in the log.

Using this hash value (**MD5: 69630e4574ec6798239b091cda43dca0)**, the VirusTotal API identified the file as malware in the next step.

Why is this showing up in SIEM?

This is visible because of Wazuh's Syscheck (FIM) module.

- We have configured the VM-K8S agent to monitor the /root directory.
- The Linux kernel has a subsystem called inotify that watches for file changes. When you issue the command marked below, the Wazuh agent tells the kernel, "**Watch the /root folder using inotify at all times**."

```
<syscheck>
  <disabled>no</disabled>
  <frequency>43200</frequency>
  <scan_on_start>yes</scan_on_start>
  <directories realtime="yes" check_all="yes">/root</directories>
```

- Whenever we created the eicar.com file, inotify immediately notified the Wazuh agent. The agent then encrypted that information (file name, time, who created it) and sent it to the manager.

The "Why" & "How":

Why: This is possible because of Wazuh's Syscheck (FIM) module. The Linux kernel has a subsystem called inotify that watches for any changes (Create, Modify, Delete) to the file system. Whenever we created the eicar.com file, inotify immediately sent a signal to the Wazuh agent, and the agent sent that information to the manager.

```
hasib@VM-K8S:~$ sudo -i
root@VM-K8S:~#
```

```
root@VM-K8S:~# set +H
```

The above command turns off history expansion, making it easy to use as plain text.

```
root@VM-K8S:~# echo 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' > /root/eicar.com
```

```
root@VM-K8S:~# exit
logout
```

How: In the agent's ossec.conf file, we configured the /root directory for real-time monitoring as follows:

```
    <directories realtime="yes">/root</directories>
```

Critical Finding:

The most important part of the report is the speed of the system. The collected logs show:

1. File Added (Rule 554): The file was added at 01:13:45 PM.

2. Active Response (Malware Removal): at 01:13:52 PM (obtained from active-responses.log).

Conclusion: The system detected a new file in just 7 seconds, verified it as malware, and automatically removed it. This is evidence of a very high level of Automated Threat Mitigation.

Device Connectivity:

Device Name	Type	IP Address	Log Source
deploy-24	SIEM Manager	192.168.145.146	Internal Logs
VM-K8S	Linux Endpoint	192.168.145.144	auth.log, syslog
Windows-Host	Windows Endpoint	DHCP	Event Viewer (Security)
Network Sim	Network Device	192.168.145.146	Syslog (UDP 514)

Task 3: SIEM Use Cases Implementation

Task 3.a: Demonstrate how to block malicious IP addresses from accessing web resources

on a web server. To do this ,you will set up your web servers on select endpoints

within your infrastructure, and try to access them from an external endpoint.

Solution:

Objective:

The main objective of this lab is to build an Active Defense system in an enterprise environment. Traditional monitoring systems only detect attacks, but here, using Wazuh SIEM and Active

Response, the process of identifying malicious IPs and automatically blocking them in the firewall has been completed.

Infrastructure Stabilization:

Before starting the lab, we solved some critical technical issues with the system:

Disk Recovery & Space Management: Services were being disrupted as the disk on the manager node deploy-24 reached 100% full. We freed up space by running apt-get clean and truncating the Owazuh log.

```
hasib@deploy-24:~$ sudo growpart /dev/sda 2
[sudo] password for hasib:
CHANGED: partition=2 start=4096 old: size=71299039 end=71303134 new: size=104853
471 end=104857566
hasib@deploy-24:~$ sudo resize2fs /dev/sda2
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/sda2 is mounted on /; on-line resizing required
old_desc_blocks = 5, new_desc_blocks = 7
The filesystem on /dev/sda2 is now 13106683 (4k) blocks long.
```

Version Mismatch Correction: The connection was broken due to a mismatch between the manager version (v4.14.3) and the dashboard version (v4.7.5). We updated the entire stack to v4.14.3 and synchronized it.

```
hasib@deploy-24:~$ sudo /var/ossec/bin/wazuh-control info
[sudo] password for hasib:
WAZUH_VERSION="v4.14.3"
WAZUH_REVISION="rc3"
WAZUH_TYPE="server"
```

SSL & Certificate Fix: The dashboard was not running on port 443 due to missing SSL certificate. The dashboard was restored by copying the certificate from the indexer and setting permissions (chmod 640) and ownership.

```
hasib@deploy-24:~$ sudo chmod 755 /etc/wazuh-dashboard/certs
hasib@deploy-24:~$ sudo chmod 640 /etc/wazuh-dashboard/certs/dashboard.pem
sudo chmod 640 /etc/wazuh-dashboard/certs/dashboard-key.pem
sudo chmod 640 /etc/wazuh-dashboard/certs/root-ca.pem
hasib@deploy-24:~$ sudo chown -R wazuh-dashboard:wazuh-dashboard /etc/wazuh-dashboard/certs
```

Then the dashboard was restarted.

```
hasib@deploy-24:~$ sudo systemctl restart wazuh-dashboard
```

Verification of Dashboard Port:

Then check if the port is listening:

```
hasib@deploy-24:~$ sudo ss -tulpn | grep 443
tcp   LISTEN 0      511                0.0.0.0:443        0.0.0.0:*    users:(("node",pid=14774,fd=19))
```

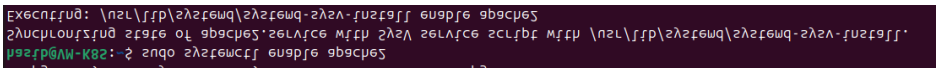
Web Server and Monitoring Setup (Victim: VM-K8S):

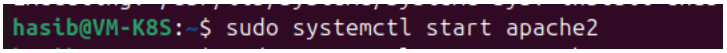
I have prepared an Ubuntu machine named VM-K8S as the target endpoint.

Apache2 Web Server Configuration:

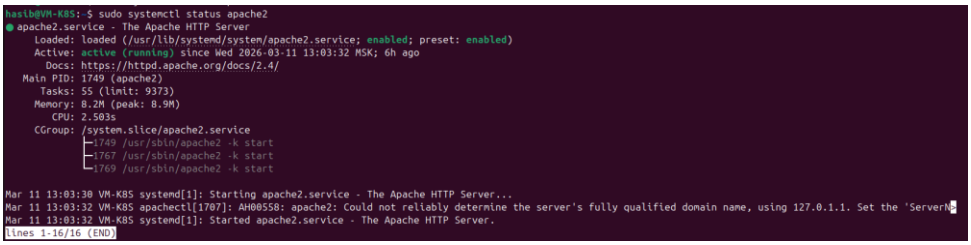
Apache has been installed on the server and status has been checked:





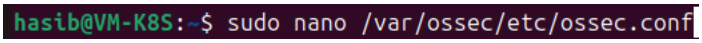


After checking the status, it was found to be Active (running).



Log Monitoring Integration:

The Owazuh agent has been instructed to read the access logs of the Apache server. For this, the following block has been added to the /var/ossec/etc/ossec.conf file of the endpoint:




```
  <localfile>
  <log_format>apache</log_format>
  <location>/var/log/apache2/access.log</location>
</localfile>
```

Active Response Configuration (Manager: deploy-24)

We have set up the Manager in a logical chain or 'if-then' condition. If specific malicious activity is detected from an IP, the Manager will automatically send commands to the endpoint.

Active Response Block (Manager Configuration):

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5710, 5712, 31101, 31103, 31106, 31106, 31151</rules_id>
  <timeout>600</timeout>
</active-response>
```

Rule 31101: 4000 level error (Attack Pattern) on web server. Attempts to access sensitive files (such as `/etc/passwd`).

Timeout: The malicious IP is temporarily blocked for 600 seconds or 10 minutes.

Attack Simulation and Security Event Analysis

We ran a scripted directory traversal attack to access sensitive files (/etc/passwd) on VM-K8S from deploy-24.

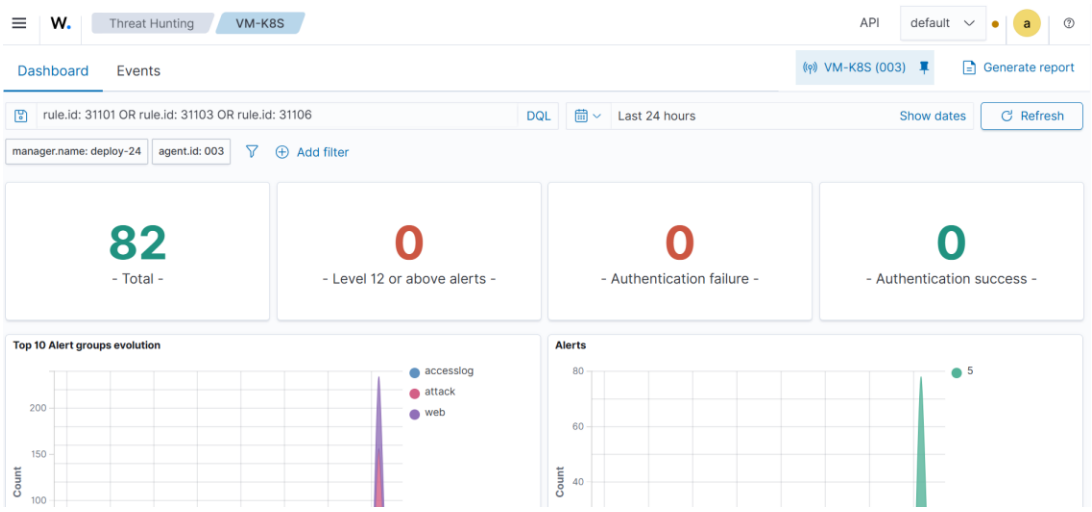
```
hasib@deploy-24:~$ for i in {1..10}; do curl -s -o /dev/null "http://192.168.145.144/etc/passwd"; echo "Aggressive Attack $i sent"; sleep 1; done

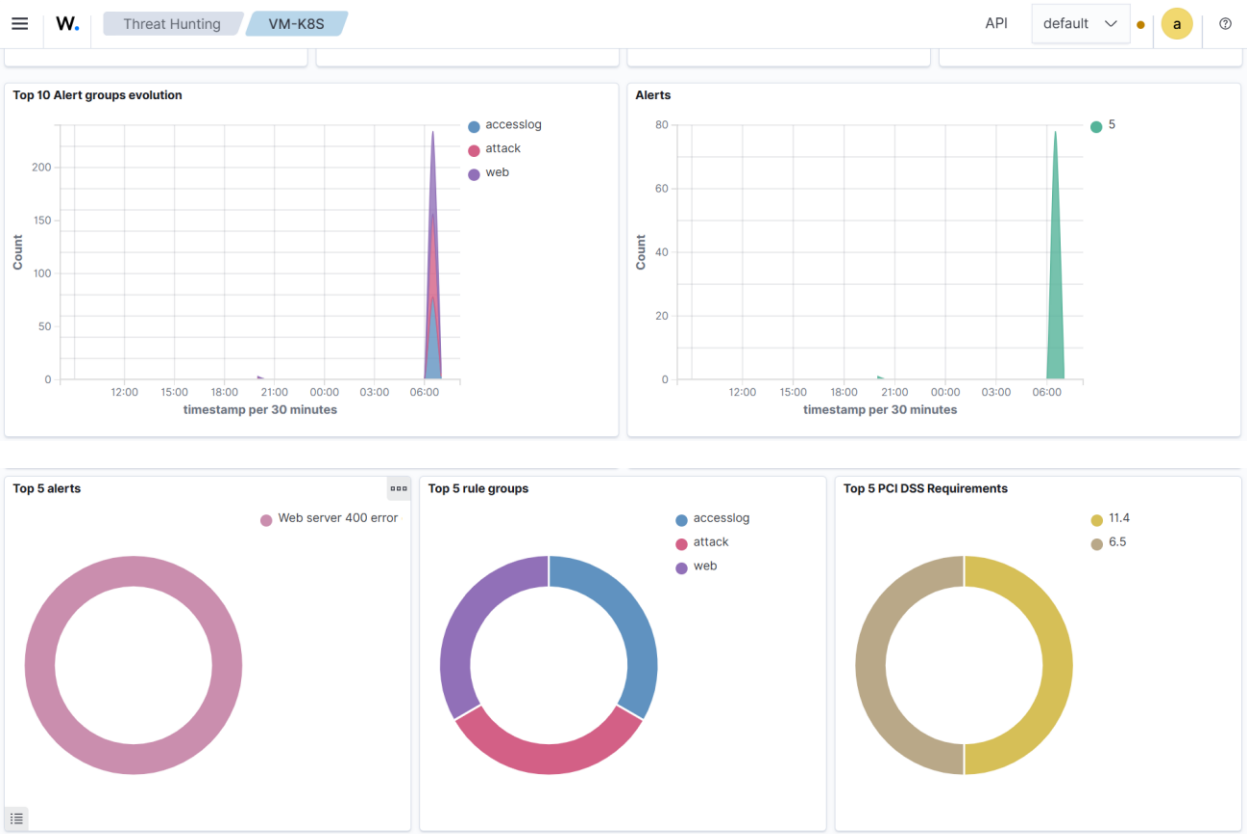
Aggressive Attack 1 sent
Aggressive Attack 2 sent
Aggressive Attack 3 sent
Aggressive Attack 4 sent
Aggressive Attack 5 sent
Aggressive Attack 6 sent
Aggressive Attack 7 sent
Aggressive Attack 8 sent
Aggressive Attack 9 sent
Aggressive Attack 10 sent
```

Dashboard & Events Analysis:

During the attack, 82 hits were found on the dashboard. The SIEM identified each malicious request separately, then ran the firewall-drop script.

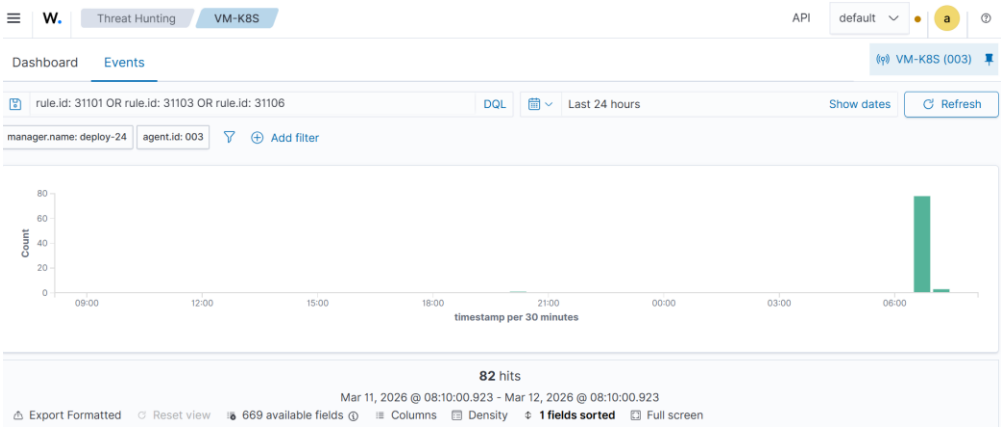
Rule 31101 - Web server 400 error code:





Security Event Analysis:

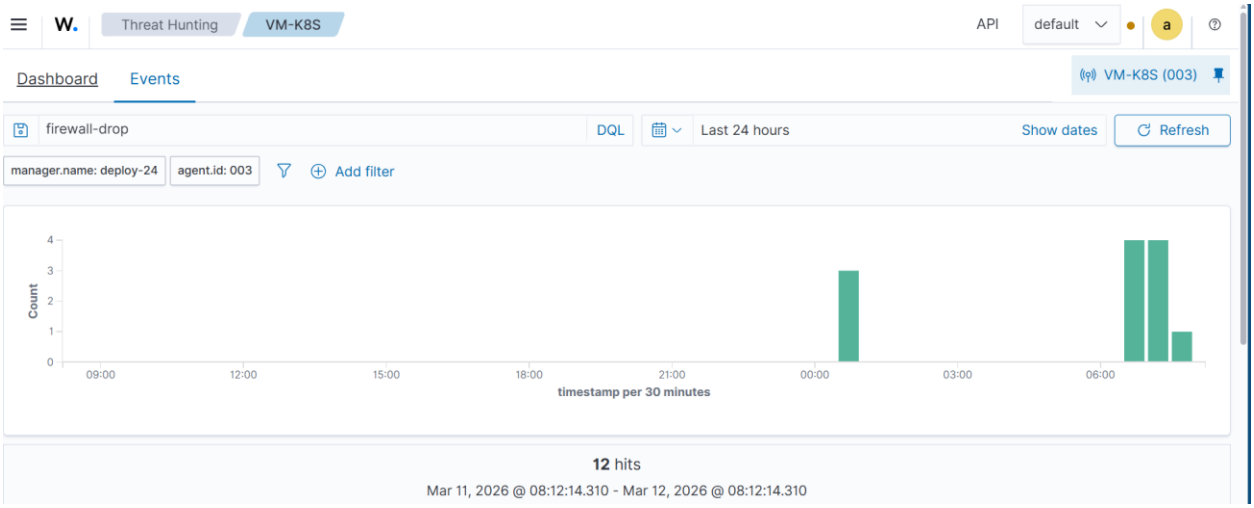
Security Event 31101 - Web server 400 error code: This is a Level 5 (Low-medium severity) alert. It is triggered when a user sends a URL request to our victim server (VM-K8S) that does not exist on the server (404) or that carries a suspicious pattern.



In this case, the attack attempted to access the /etc/passwd file. Wazuh's signature database identified this pattern as a 'Directory Traversal' attack and fired **rule 31101**.

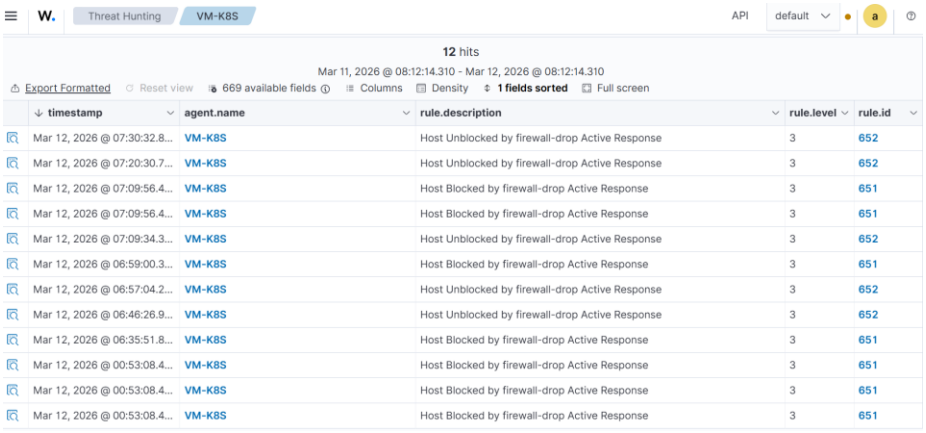
timestamp	agent.name	rule.description	rule.level	rule.id
Mar 12, 2026 @ 07:20:30.8...	VM-K8S	Web server 400 error code.	5	31101
Mar 12, 2026 @ 07:09:54.4...	VM-K8S	Web server 400 error code.	5	31101
Mar 12, 2026 @ 07:09:54.4...	VM-K8S	Web server 400 error code.	5	31101
Mar 12, 2026 @ 06:59:00.2...	VM-K8S	Web server 400 error code.	5	31101
Mar 12, 2026 @ 06:58:58.2...	VM-K8S	Web server 400 error code.	5	31101
Mar 12, 2026 @ 06:58:58.2...	VM-K8S	Web server 400 error code.	5	31101
Mar 12, 2026 @ 06:58:56.3...	VM-K8S	Web server 400 error code.	5	31101
Mar 12, 2026 @ 06:58:56.2...	VM-K8S	Web server 400 error code.	5	31101
Mar 12, 2026 @ 06:57:36.2...	VM-K8S	Web server 400 error code.	5	31101
Mar 12, 2026 @ 06:57:36.2...	VM-K8S	Web server 400 error code.	5	31101
Mar 12, 2026 @ 06:57:34.2...	VM-K8S	Web server 400 error code.	5	31101
Mar 12, 2026 @ 06:57:34.2...	VM-K8S	Web server 400 error code.	5	31101
Mar 12, 2026 @ 06:57:32.2...	VM-K8S	Web server 400 error code.	5	31101
Mar 12, 2026 @ 06:57:32.2...	VM-K8S	Web server 400 error code.	5	31101

Rule 651 - Active Response (Host Blocked): This is the most important part of this lab. Filtering on the Events tab shows that the manager automatically ran the firewall-drop script due to repeated triggering of **rule 31101**.



Rule ID 651: Indicates that the host was successfully blocked.

Rule ID 652: Indicates that the host was unblocked again after the specified timeout period.

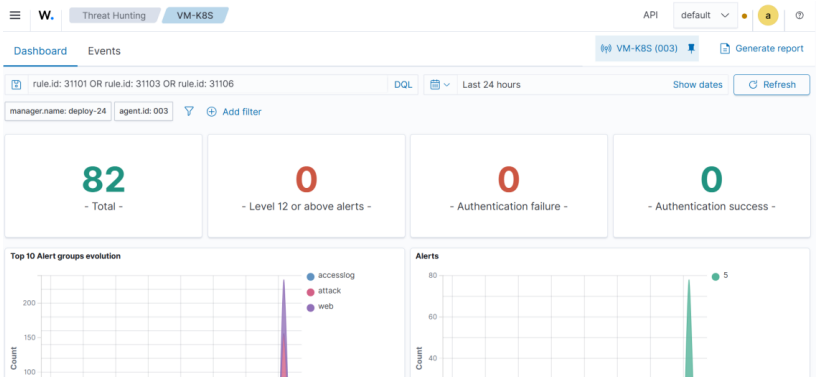


timestamp	agent.name	rule.description	rule.level	rule.id
Mar 12, 2026 @ 07:30:32.8...	VM-K8S	Host Unblocked by firewall-drop Active Response	3	652
Mar 12, 2026 @ 07:20:30.7...	VM-K8S	Host Unblocked by firewall-drop Active Response	3	652
Mar 12, 2026 @ 07:09:56.4...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651
Mar 12, 2026 @ 07:09:56.4...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651
Mar 12, 2026 @ 07:09:34.3...	VM-K8S	Host Unblocked by firewall-drop Active Response	3	652
Mar 12, 2026 @ 06:59:00.3...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651
Mar 12, 2026 @ 06:57:04.2...	VM-K8S	Host Unblocked by firewall-drop Active Response	3	652
Mar 12, 2026 @ 06:46:26.9...	VM-K8S	Host Unblocked by firewall-drop Active Response	3	652
Mar 12, 2026 @ 06:35:51.8...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651
Mar 12, 2026 @ 00:53:08.4...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651
Mar 12, 2026 @ 00:53:08.4...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651
Mar 12, 2026 @ 00:53:08.4...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651

Evidence Collection:

1. Visual Evidence:

The "**Threat Hunting**" section shows that 82 alerts were hit at a certain point during the attack. This 'spike' in the graph proves that this was not a simple user error, but rather a systematic attack.



2. Firewall Verification: By running the iptables command on the victim machine (VM-K8S), it was found that the attacker's IP (192.168.145.146) was directly added to the DROP list.

```
hasib@VM-K8S:~$ sudo iptables -L -n | grep 192.168.145.146
[sudo] password for hasib:
DROP       0    --  192.168.145.146      0.0.0.0/0
DROP       0    --  192.168.145.146      0.0.0.0/0
```

System Logs:

The log below shows how the system worked step by step.

Timestamp 07:20:31: The attacker IP (192.168.145.146) was added to the firewall (command: add).

Timestamp 07:30:32: Exactly 10 minutes later (according to the timeout policy), the system removed the IP (command: delete). This proves that the automation loop is working perfectly.

```
hasib@VM-K8S:~$ sudo tail /var/ossec/logs/active-responses.log
2026/03/12 07:20:31 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"node01","module":"wazuh-execd"},"command":"add","parameters":{"extra_args":[]
,"alert":{"timestamp":"2026-03-12T07:20:30.810+0300","rule":{"level":5,"description":"Web server 400 error code.","id":"31101","firedtimes":3,"mail":false,"groups"
:["web","accesslog","attack"],"pci_dss":["6.5","11.4"],"gdpr":["IV_35.7.d"],"nist_800_53":["SA.11","SI.4"],"tsc":["CC6.6","CC7.1","CC8.1","CC6.1","CC6.8","CC7.2","
CC7.3"]},"agent":{"id":"003","name":"VM-K8S","ip":"192.168.145.144"},"manager":{"name":"deploy-24"},"id":"1773289230.7015353","full_log":"192.168.145.146 - - [12/M
ar/2026:07:20:06 +0300] \"GET /etc/passwd HTTP/1.1\" 404 438 \"-\" \"curl/8.5.0\"","decoder":{"name":"web-accesslog"},"data":{"protocol":"GET","srcip":"192.168.145
.146","id":"404","url":"/etc/passwd"},"location":"/var/log/apache2/access.log"},"program":"active-response/bin/firewall-drop"}}

2026/03/12 07:20:31 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"firewall-drop","module":"active-response"},"command":"check_keys","parameters
":{"keys":["192.168.145.146"]}}
2026/03/12 07:20:31 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"node01","module":"wazuh-execd"},"command":"continue","parameters":{"extra_arg
s":[],"alert":{"timestamp":"2026-03-12T07:20:30.810+0300","rule":{"level":5,"description":"Web server 400 error code.","id":"31101","firedtimes":3,"mail":false,"gr
oups":["web","accesslog","attack"],"pci_dss":["6.5","11.4"],"gdpr":["IV_35.7.d"],"nist_800_53":["SA.11","SI.4"],"tsc":["CC6.6","CC7.1","CC8.1","CC6.1","CC6.8","CC7
.2","CC7.3"]},"agent":{"id":"003","name":"VM-K8S","ip":"192.168.145.144"},"manager":{"name":"deploy-24"},"id":"1773289230.7015353","full_log":"192.168.145.146 - -
[12/Mar/2026:07:20:06 +0300] \"GET /etc/passwd HTTP/1.1\" 404 438 \"-\" \"curl/8.5.0\"","decoder":{"name":"web-accesslog"},"data":{"protocol":"GET","srcip":"192.16
8.145.146","id":"404","url":"/etc/passwd"},"location":"/var/log/apache2/access.log"},"program":"active-response/bin/firewall-drop"}}

2026/03/12 07:20:31 active-response/bin/firewall-drop: Ended
2026/03/12 07:30:32 active-response/bin/firewall-drop: Starting
2026/03/12 07:30:32 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"node01","module":"wazuh-execd"},"command":"delete","parameters":{"extra_args"
:[],"alert":{"timestamp":"2026-03-12T07:20:30.810+0300","rule":{"level":5,"description":"Web server 400 error code.","id":"31101","firedtimes":3,"mail":false,"grou
ps":["web","accesslog","attack"],"pci_dss":["6.5","11.4"],"gdpr":["IV_35.7.d"],"nist_800_53":["SA.11","SI.4"],"tsc":["CC6.6","CC7.1","CC8.1","CC6.1","CC6.8","CC7.2
","CC7.3"]},"agent":{"id":"003","name":"VM-K8S","ip":"192.168.145.144"},"manager":{"name":"deploy-24"},"id":"1773289230.7015353","full_log":"192.168.145.146 - - [1
2/Mar/2026:07:20:06 +0300] \"GET /etc/passwd HTTP/1.1\" 404 438 \"-\" \"curl/8.5.0\"","decoder":{"name":"web-accesslog"},"data":{"protocol":"GET","srcip":"192.168.
145.146","id":"404","url":"/etc/passwd"},"location":"/var/log/apache2/access.log"},"program":"active-response/bin/firewall-drop"}}

2026/03/12 07:30:32 active-response/bin/firewall-drop: Ended
```

b. Simulate a brute force attack against your infrastructure and demonstrate how you would detect the attack on each of the devices within your infrastructure. Are you able to detect the attack? If not, ensure you are able to.

Solution:

Simulating brute force attacks on different layers of the entire infrastructure (Linux, Windows, Web, and Network) and successfully detecting those attacks through Wazuh SIEM.

1. Simulation Phase (The Commands Used)

In this phase, we have launched attacks on different endpoints from the Manager/Attacker machine. Below are the commands used for each device and their output:

Web Infrastructure (Apache Server)

A brute force attack has been simulated by sending repeated requests to the login page on the web server.

Target IP: 192.168.145.144

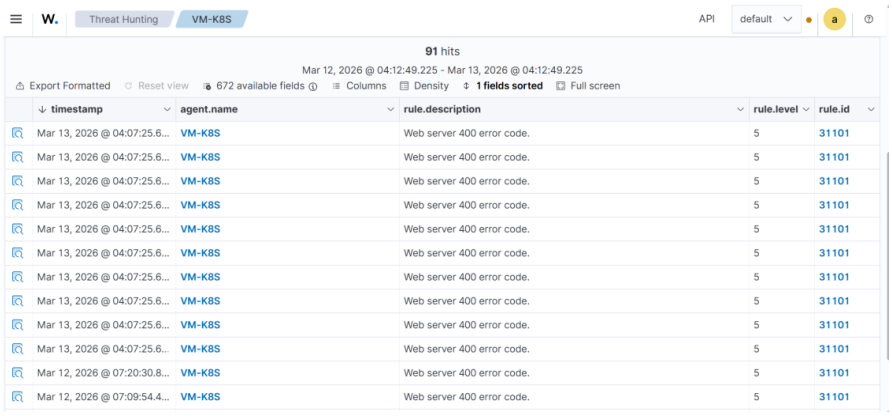
```
hasib@deploy-24:~$ for i in {1..10}; do curl "http://192.168.145.144/login?user=admin"; done
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.145.144 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.145.144 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.145.144 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
```

```
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.145.144 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.145.144 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.145.144 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
```

```
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.145.144 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.145.144 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.145.144 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.145.144 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
```

```
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.145.144 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.145.144 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.145.144 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.145.144 Port 80</address>
</body></html>
```

Repeated 404 Not Found responses were received from the server, which triggered **Rule 31101 (Web server 400 error code)** in the Owazuh dashboard.

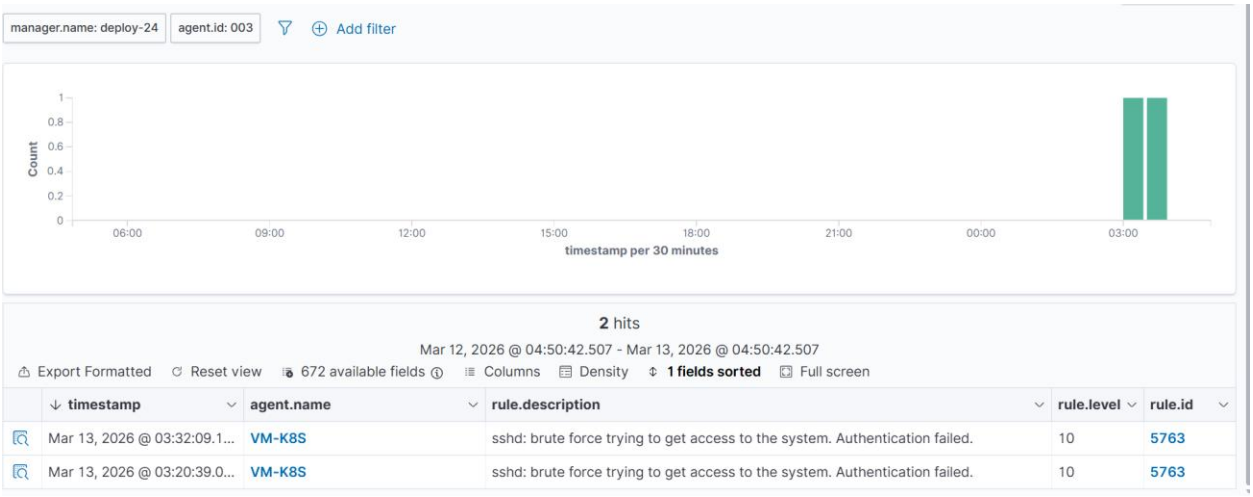


Agent 003 (Linux Endpoint): This command is used to perform Linux endpoint SSH brute force.

Target IP: 192.168.145.144

```
hasib@deploy-24:~$ for i in {1..10}; do sshpass -p "Wrong123" ssh -o StrictHostKeyChecking=no hasib@192.168.145.144; done
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
```

The output always shows a Permission denied message. This is detected as **Rule 5783 (SSH brute force)** in Owazuh.



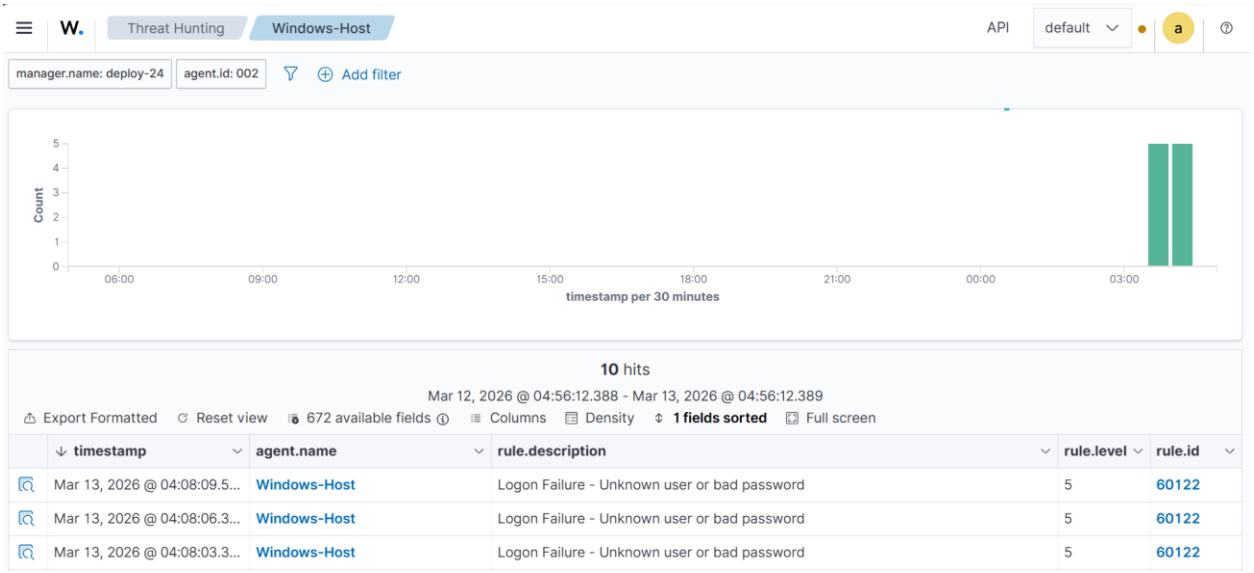
Windows-Host (Windows Endpoint):

An invalid login attempt was made using Remote Desktop Protocol (RDP) on a Windows machine.

Target IP: 192.168.145.1

```
hasib@deploy-24:~$ xfreerdp /u:Admin /p:Pass /v:192.168.145.1 /cert:ignore
[04:07:54:400] [31831:31832] [WARN][com.freerdp.core.nla] - SPNEGO received NTSTATUS: STATUS_LOGON_FAILURE [0xC000006D] from server
[04:07:54:400] [31831:31832] [ERROR][com.freerdp.core] - nla_recv_pdu:freerdp_set_last_error_ex ERRCONNECT_LOGON_FAILURE [0x00020014]
[04:07:54:400] [31831:31832] [ERROR][com.freerdp.core.rdp] - rdp_recv_callback: CONNECTION_STATE_NLA - nla_recv_pdu() fail
[04:07:54:400] [31831:31832] [ERROR][com.freerdp.core.transport] - transport_check_fds: transport->ReceiveCallback() - -1
```

The output shows STATUS_LOGON_FAILURE. This creates **Event ID 4625** in the Windows Event Log and triggers **Rule 60122** in Owazuh.



Network Sim (Network Device):

A remote syslog was sent to simulate an error message from a network device.

Manager IP: 192.168.145.146

```
hasib@deploy-24:~$ logger -n 192.168.145.146 -P 514 "NetworkSim error: Multiple failed login attempts from 192.168.145.50 on port 22"
```

The syslog is sent directly to port 514 of Wazuh Manager, which ensures network security monitoring

2. Dashboard Verification

After completing the simulation phase, evidence of the attacks being detected was collected using Wazuh Dashboard (deploy-24). The detailed steps are given below:

Dashboard Navigation

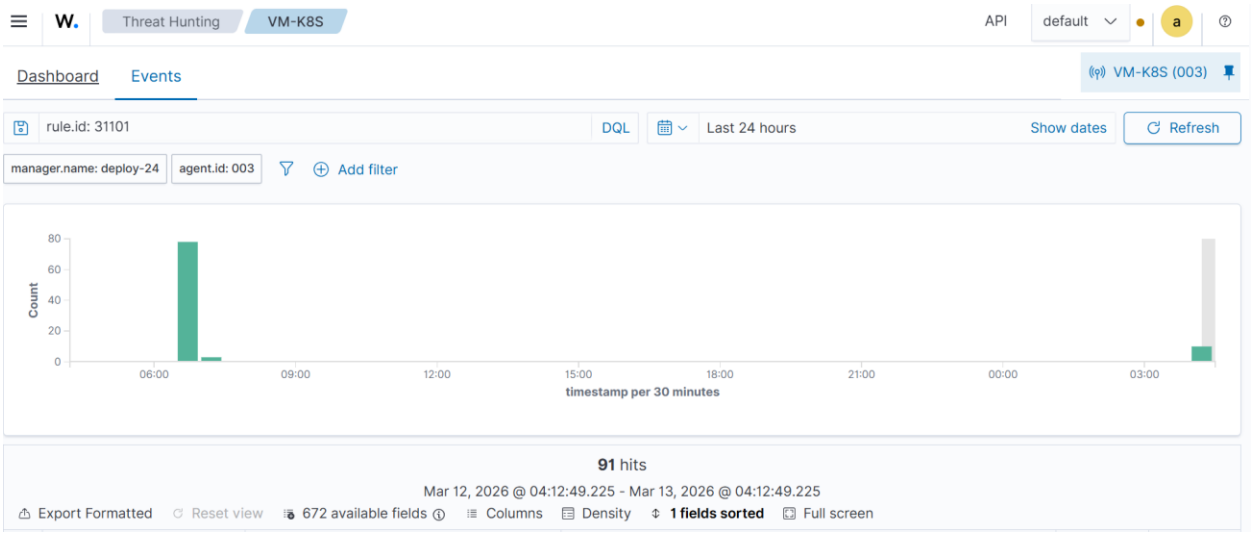
After going to Threat Hunting option from the left side menu of Wazuh web interface, Events was selected from the top tab. Then the time was set to Last 24 hours.

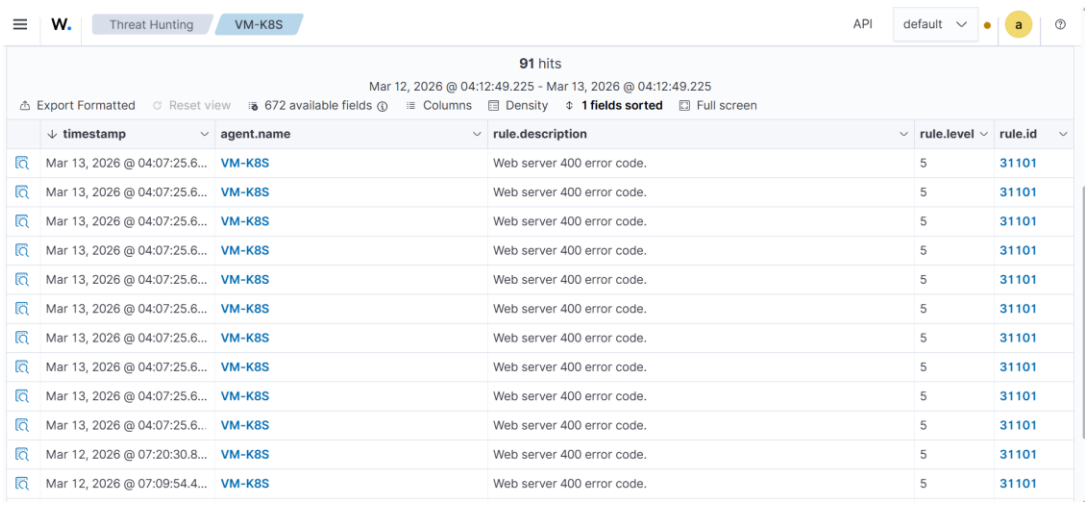
Event Filtering and Analysis (DQL Filters):

A. Web Infrastructure (Apache) Detection

We identified the curl attack using the following filter in the search bar:

Filter: rule.id: 3110





timestamp	agent.name	rule.description	rule.level	rule.id
Mar 13, 2026 @ 04:07:25.6...	VM-K8S	Web server 400 error code.	5	31101
Mar 13, 2026 @ 04:07:25.6...	VM-K8S	Web server 400 error code.	5	31101
Mar 13, 2026 @ 04:07:25.6...	VM-K8S	Web server 400 error code.	5	31101
Mar 13, 2026 @ 04:07:25.6...	VM-K8S	Web server 400 error code.	5	31101
Mar 13, 2026 @ 04:07:25.6...	VM-K8S	Web server 400 error code.	5	31101
Mar 13, 2026 @ 04:07:25.6...	VM-K8S	Web server 400 error code.	5	31101
Mar 13, 2026 @ 04:07:25.6...	VM-K8S	Web server 400 error code.	5	31101
Mar 13, 2026 @ 04:07:25.6...	VM-K8S	Web server 400 error code.	5	31101
Mar 13, 2026 @ 04:07:25.6...	VM-K8S	Web server 400 error code.	5	31101
Mar 13, 2026 @ 04:07:25.6...	VM-K8S	Web server 400 error code.	5	31101
Mar 12, 2026 @ 07:20:30.8...	VM-K8S	Web server 400 error code.	5	31101
Mar 12, 2026 @ 07:09:54.4...	VM-K8S	Web server 400 error code.	5	31101

JSON:

Table JSON


```
{
  "_index": "wazuh-alerts-4.x-2026.03.13",
  "_id": "LDW75JwBog-E-RmF_eNY",
  "_score": null,
  "_source": {
    "input": {
      "type": "log"
    },
    "agent": {
      "ip": "192.168.145.144",
      "name": "VM-K8S",
      "id": "003"
    },
    "manager": {
      "name": "deploy-24"
    },
    "data": {
      "protocol": "GET",
      "srcip": "192.168.145.146",
      "id": "404",
      "url": "/login?user=admin"
    },
    "rule": {
      "firedtimes": 10,
      "mail": false,
      "level": 5,
      "pci_dss": [
        "6.5",
        "11.4"
      ],
      "tsc": [
        "CC6.6",
        "CC7.1",
        "CC8.1",
        "CC6.1",
        "CC6.8",
        "CC7.2",
        "CC7.3"
      ],
      "description": "Web server 400 error code.",
      "groups": [
        "web",
        "accesslog",
        "attack"
      ],
      "id": "31101",
      "nist_800_53": [
        "SA.11",
        "SI.4"
      ],
      "gdpr": [
        "IV_35.7.d"
      ]
    },
    "location": "/var/log/apache2/access.log",
    "decoder": {
      "name": "web-accesslog"
    },
    "id": "1773364045.324507",
    "full_log": "192.168.145.146 - - [13/Mar/2026:04:07:33 +0300] \"GET /login?user=admin HTTP/1.1\" 404 438 \"-\" \"curl/8.5.0\"",
    "timestamp": "2026-03-13T04:07:25.669+0300"
  },
  "fields": {
    "timestamp": [
      "2026-03-13T01:07:25.669Z"
    ]
  },
  "sort": [
    1773364045669
  ]
}
```

Key Evidence:

- **srcip (Attacker):** The attack came from 192.168.145.146 (deploy-24).
- **firedtimes:** 10 times (since 10 requests were sent in the loop).
- **URL:** /login?user=admin (the attack was carried out on this specific URL).

B. Linux Endpoint (SSH Brute Force) Verification:

SSH Brute Force attack executed on Linux endpoint (VM-K8S) to ensure server security has been successfully detected.

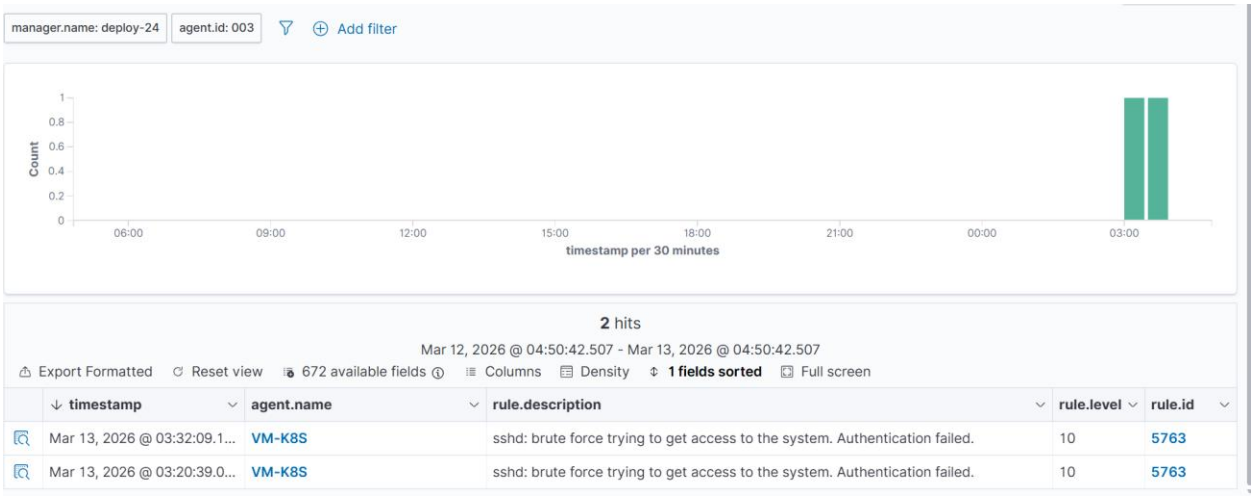
Searching and Filtering (DQL Filter)

The following filter has been applied by going to the Threat Hunting section of Wazuh dashboard:

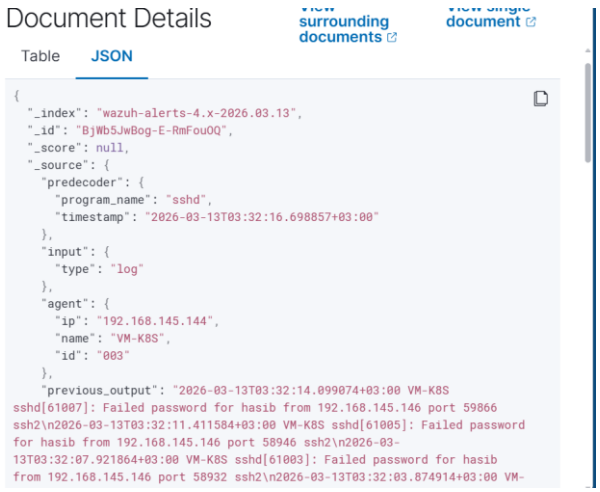
Filter: rule.id: 5763

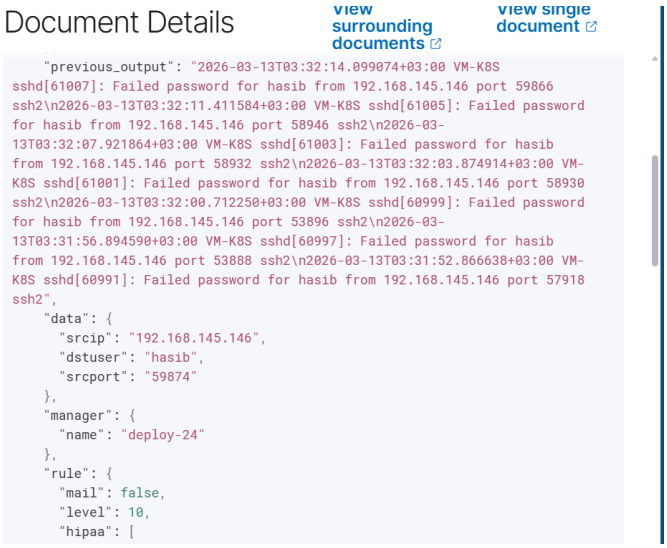
Detection results:

After filtering, the "**sshd: brute force trying to get access to the system**" alert is displayed on the dashboard. This is a Level 10 (High Priority) alert, indicating that a major intrusion attempt has been made to the system.

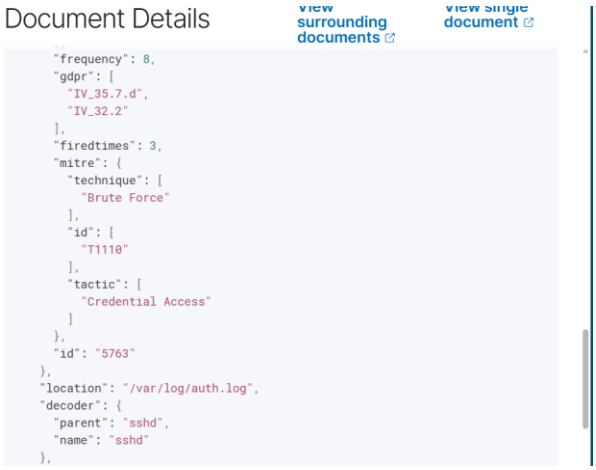


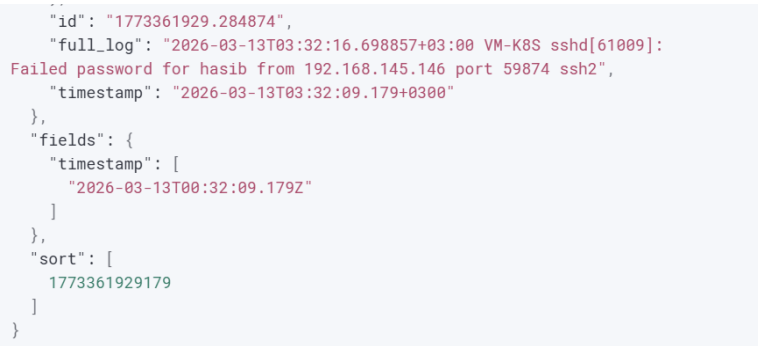
JSON:


Document Details
View surrounding documents
View single document
Table
JSON
{
"_index": "wazuh-alerts-4.x-2026.03.13",
"_id": "BjWb5JwBog-E-RmFouOQ",
"_score": null,
"_source": {
"predecoder": {
"program_name": "sshd",
"timestamp": "2026-03-13T03:32:16.698857+03:00"
},
"input": {
"type": "log"
},
"agent": {
"ip": "192.168.145.144",
"name": "VM-K8S",
"id": "003"
},
"previous_output": "2026-03-13T03:32:14.099074+03:00 VM-K8S sshd[61007]: Failed password for hasib from 192.168.145.146 port 59866 ssh2\n2026-03-13T03:32:11.411584+03:00 VM-K8S sshd[61005]: Failed password for hasib from 192.168.145.146 port 58946 ssh2\n2026-03-13T03:32:07.921864+03:00 VM-K8S sshd[61003]: Failed password for hasib from 192.168.145.146 port 58932 ssh2\n2026-03-13T03:32:03.874914+03:00 VM-


Document Details
View surrounding documents
View single document
"previous_output": "2026-03-13T03:32:14.099074+03:00 VM-K8S sshd[61007]: Failed password for hasib from 192.168.145.146 port 59866 ssh2\n2026-03-13T03:32:11.411584+03:00 VM-K8S sshd[61005]: Failed password for hasib from 192.168.145.146 port 58946 ssh2\n2026-03-13T03:32:07.921864+03:00 VM-K8S sshd[61003]: Failed password for hasib from 192.168.145.146 port 58932 ssh2\n2026-03-13T03:32:03.874914+03:00 VM-K8S sshd[61001]: Failed password for hasib from 192.168.145.146 port 58930 ssh2\n2026-03-13T03:32:00.712250+03:00 VM-K8S sshd[60999]: Failed password for hasib from 192.168.145.146 port 53896 ssh2\n2026-03-13T03:31:56.894590+03:00 VM-K8S sshd[60997]: Failed password for hasib from 192.168.145.146 port 53888 ssh2\n2026-03-13T03:31:52.866638+03:00 VM-K8S sshd[60991]: Failed password for hasib from 192.168.145.146 port 57918 ssh2",
"data": {
"srcip": "192.168.145.146",
"dstuser": "hasib",
"srcport": "59874"
},
"manager": {
"name": "deploy-24"
},
"rule": {
"mail": false,
"level": 10,
"hipaa": [


"hipaa": [
"164.312.b"
],
"pci_dss": [
"11.4",
"10.2.4",
"10.2.5"
],
"tsc": [
"CC6.1",
"CC6.8",
"CC7.2",
"CC7.3"
],
"description": "sshd: brute force trying to get access to the system. Authentication failed.",
"groups": [
"syslog",
"sshd",
"authentication_failures"
],
"nist_800_53": [
"SI.4",
"AU.14",
"AC.7"
],





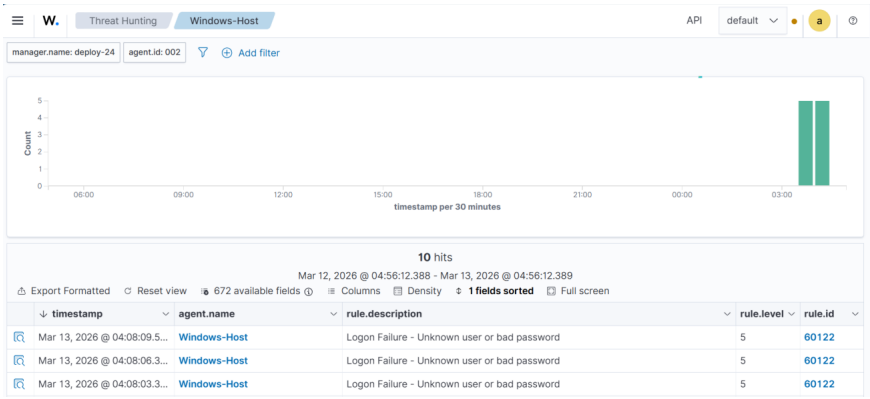
C. Windows Endpoint (RDP/Network Logon Failure) Verification:

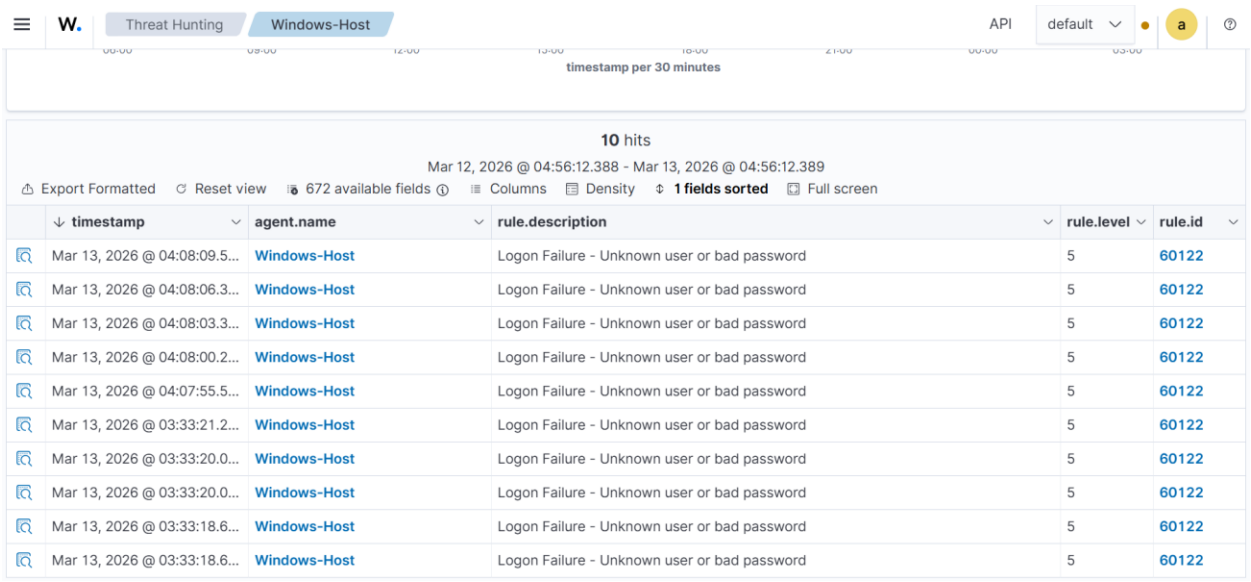
Security events resulting from invalid login attempts on the infrastructure Windows host (Windows-Host) are successfully detected in the central dashboard by the Wazuh Agent.

Searching and Filtering (DQL Filter):

The following filter is used to find the specific Windows alert by going to the Threat Hunting tab of the Wazuh dashboard:

Filter: rule.id: 60122

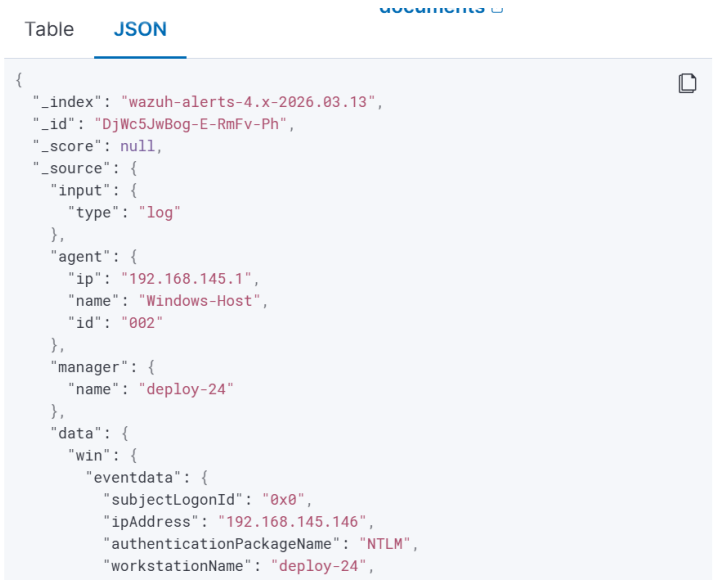




Detection Results:

After filtering, the "**Logon Failure - Unknown user or bad password**" alert is displayed on the dashboard. This proves that Owazuh is able to successfully read Windows' internal security audit logs.

Important JSON data as evidence (Evidence Collection):



Document Details

View surrounding documents | View single document


```
        "workstationName": "deploy-24",
        "subStatus": "0xc000006a",
        "logonProcessName": "NtLmSsp",
        "targetUserName": "Administrator",
        "keyLength": "0",
        "subjectUserSid": "S-1-0-0",
        "processId": "0x0",
        "ipPort": "0",
        "failureReason": "%%2313",
        "targetUserSid": "S-1-0-0",
        "logonType": "3",
        "status": "0xc000006d"
      },
      "system": {
        "eventID": "4625",
        "keywords": "0x8010000000000000",
        "providerGuid": "{54849625-5478-4994-a5ba-3e3b0328c30d}",
        "level": "0",
        "channel": "Security",
        "opcode": "0",
```


Document Details

View surrounding documents | View single document


```
        "opcode": "0",
        "message": "\"An account failed to log
on.\r\n\r\nSubject:\r\n\tSecurity ID:\t\tS-1-0-0\r\n\tAccount Name:\t\t-
\r\n\tAccount Domain:\t\t-\r\n\tLogon ID:\t\t0x0\r\n\r\nLogon
Type:\t\t\t3\r\n\r\nAccount For Which Logon Failed:\r\n\tSecurity
ID:\t\tS-1-0-0\r\n\tAccount Name:\t\tAdministrator\r\n\tAccount
Domain:\t\t-\r\n\r\nFailure Information:\r\n\tFailure Reason:\t\tUnknown
user name or bad password.\r\n\tStatus:\t\t\t0xC000006D\r\n\tSub
Status:\t\t0xC000006A\r\n\r\nProcess Information:\r\n\tCaller Process
ID:\t0x0\r\n\tCaller Process Name:\t-\r\n\r\nNetwork
Information:\r\n\tWorkstation Name:\tdeploy-24\r\n\tSource Network
Address:\t192.168.145.146\r\n\tSource Port:\t\t0\r\n\r\nDetailed
Authentication Information:\r\n\tLogon Process:\t\tNtLmSsp
\r\n\tAuthentication Package:\tNTLM\r\n\tTransited Services:\t-
\r\n\tPackage Name (NTLM only):\t-\r\n\tKey Length:\t\t0\r\n\r\nThis event
is generated when a logon request fails. It is generated on the computer
where access was attempted.\r\n\r\nThe Subject fields indicate the account
on the local system which requested the logon. This is most commonly a
service such as the Server service, or a local process such as
Winlogon.exe or Services.exe.\r\n\r\nThe Logon Type field indicates the
kind of logon that was requested. The most common types are 2
(interactive) and 3 (network).\r\n\r\nThe Process Information fields
indicate which account and process on the system requested the
logon.\r\n\r\nThe Network Information fields indicate where a remote logon
request originated. Workstation name is not always available and may be
```


Document Details

View surrounding documents | View single document


```
logon.\r\n\r\nThe Network Information fields indicate where a remote logon
request originated. Workstation name is not always available and may be
left blank in some cases.\r\n\r\nThe authentication information fields
provide detailed information about this specific logon request.\r\n\t-
Transited services indicate which intermediate services have participated
in this logon request.\r\n\t- Package name indicates which sub-protocol
was used among the NTLM protocols.\r\n\t- Key length indicates the length
of the generated session key. This will be 0 if no session key was
requested.\"",
        "version": "0",
        "systemTime": "2026-03-13T00:33:16.6999392Z",
        "eventRecordID": "628787",
        "threadID": "1192",
        "computer": "Hasib",
        "task": "12544",
        "processID": "1100",
        "severityValue": "AUDIT_FAILURE",
        "providerName": "Microsoft-Windows-Security-Auditing"
      }
    }
  },
  "rule": {
    "mail": false,
    "level": 5,
    "pci_dss": [
      "10.2.4",
```



4. Network Device (Network Sim) Verification:

The Syslog protocol is used to monitor logs from non-agent devices (e.g. routers or switches).

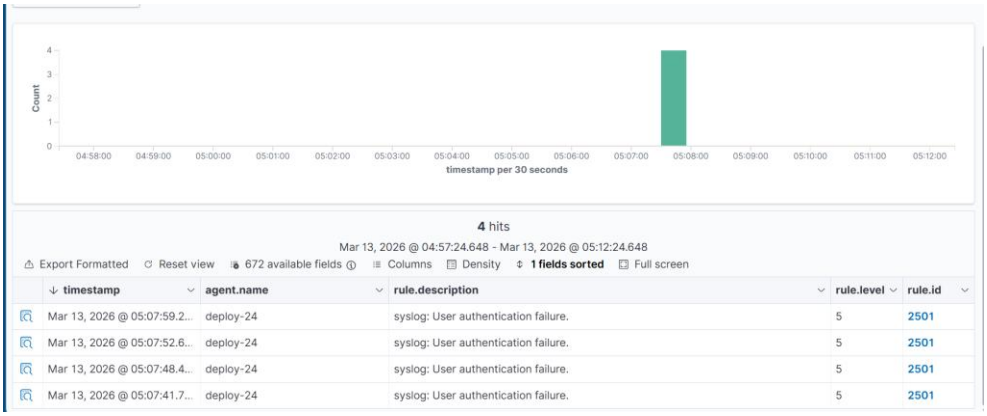
Searching and Filtering (DQL Filter)

Evidence is collected by typing NetworkSim directly in the search bar of the dashboard or by using the following filter:

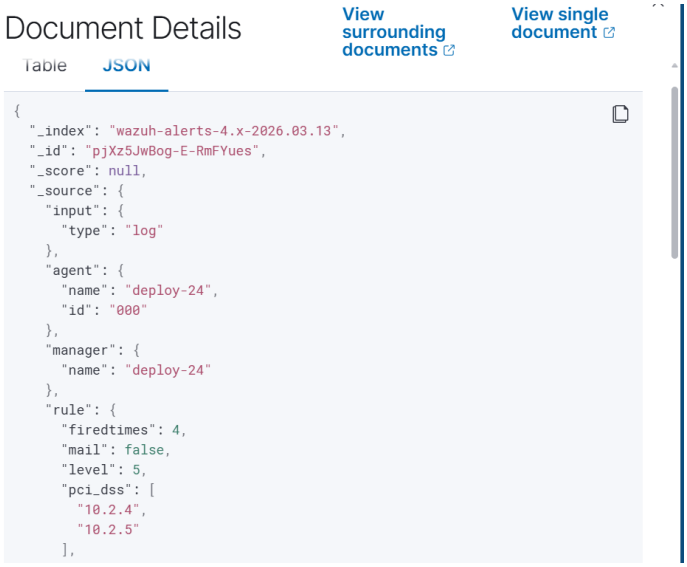
Filter: syslog: User authentication failure (rule.id: 2501).

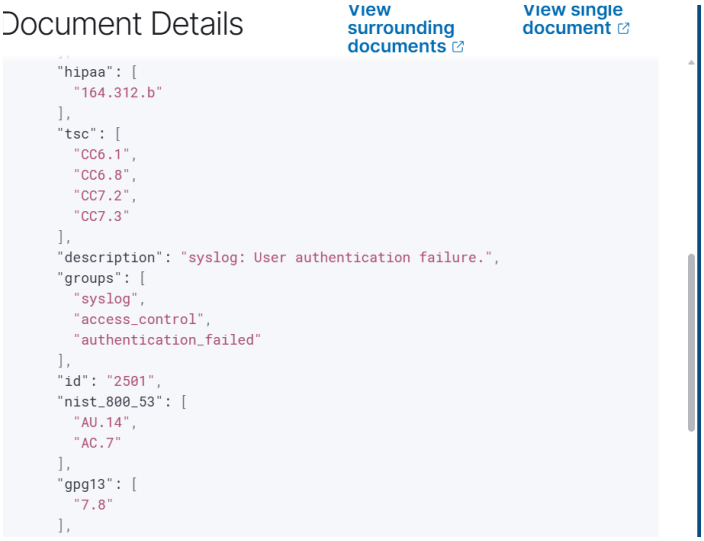
Detection Result:

The sent custom syslog message was successfully received by the Owazuh Manager.



Important JSON data as evidence:







Archive Log Verification (Manager Terminal):

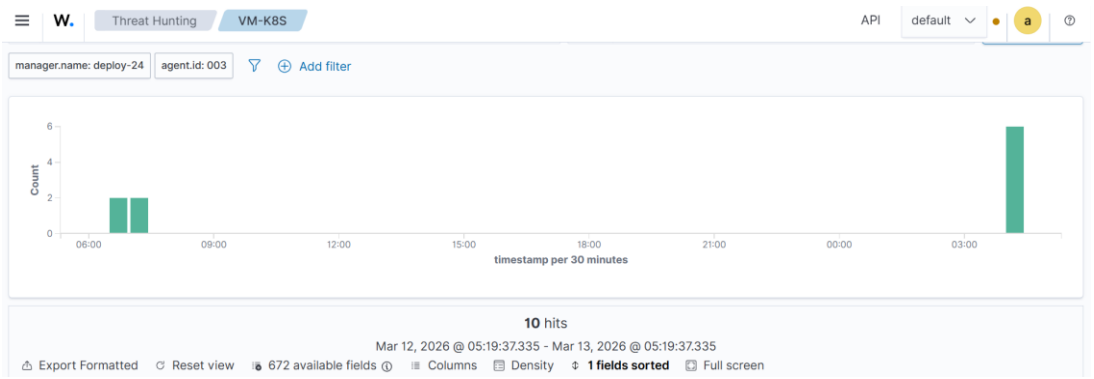
The network device logs are sent directly to the manager via UDP port 514. The output of the following command proves that the manager has received these remote logs. This is proof of non-agent network device monitoring.

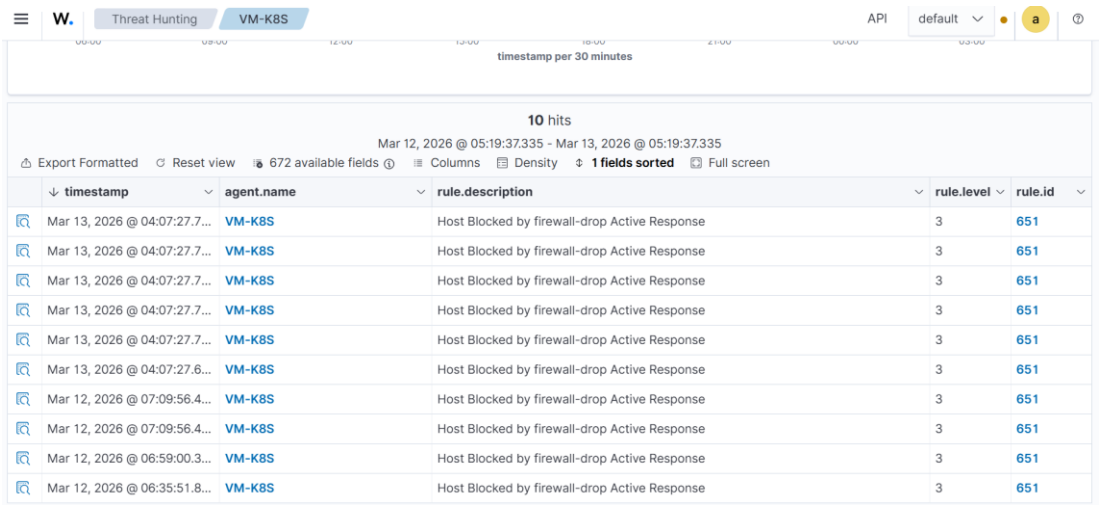


Active Response Verification:

Filter: rule.id: 651

Result: The **"Host Blocked by firewall-drop Active Response"** alert was found in the dashboard. This means that whenever our configured Owazuh Manager detected an attack, it immediately sent a command to the target endpoint (VM-K8S) to block the attacker's IP in the firewall.





timestamp	agent.name	rule.description	rule.level	rule.id
Mar 13, 2026 @ 04:07:27.7...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651
Mar 13, 2026 @ 04:07:27.7...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651
Mar 13, 2026 @ 04:07:27.7...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651
Mar 13, 2026 @ 04:07:27.7...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651
Mar 13, 2026 @ 04:07:27.7...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651
Mar 13, 2026 @ 04:07:27.6...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651
Mar 12, 2026 @ 07:09:56.4...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651
Mar 12, 2026 @ 07:09:56.4...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651
Mar 12, 2026 @ 06:59:00.3...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651
Mar 12, 2026 @ 06:35:51.8...	VM-K8S	Host Blocked by firewall-drop Active Response	3	651

JSON:



Document Details

View surrounding documents | View single document


```
    "version": "1",
    "parameters": {
      "alert": {
        "agent": {
          "ip": "192.168.145.144",
          "name": "VM-K8S",
          "id": "003"
        },
        "manager": {
          "name": "deploy-24"
        },
        "data": {
          "protocol": "GET",
          "srcip": "192.168.145.146",
          "id": "404",
          "url": "/login?user=admin"
        },
        "rule": {
          "firedtimes": "8",
          "mail": "false",
          "level": "5",
          "pci_dss": [
            "6.5",
            "11.4"
          ],
```


Document Details

View surrounding documents | View single document


```
          "tsc": [
            "CC6.6",
            "CC7.1",
            "CC8.1",
            "CC6.1",
            "CC6.8",
            "CC7.2",
            "CC7.3"
          ],
          "description": "Web server 400 error code.",
          "groups": [
            "web",
            "accesslog",
            "attack"
          ],
          "id": "31101",
          "nist_800_53": [
            "SA.11",
            "SI.4"
          ],
          "gdpr": [
            "IV_35.7.d"
          ]
        },
```


Document Details

View surrounding documents | View single document


```
        "location": "/var/log/apache2/access.log",
        "id": "1773364045.322723",
        "decoder": {
          "name": "web-accesslog"
        },
        "timestamp": "2026-03-13T04:07:25.649+0300",
        "full_log": "192.168.145.146 - - [13/Mar/2026:04:07:33 +0300]
\"GET /login?user=admin HTTP/1.1\" 404 438 \"-\" \"curl/8.5.0\""
      },
      "extra_args": [],
      "program": "active-response/bin/firewall-drop"
    },
    "command": "add"
  },
  "rule": {
    "firedtimes": 6,
    "mail": false,
    "level": 3,
    "pci_dss": [
      "11.4"
    ],
```

Document Details

View surrounding documents | View single document


```
      ],
      "tsc": [
        "CC6.1",
        "CC6.8",
        "CC7.2",
        "CC7.3",
        "CC7.4"
      ],
      "description": "Host Blocked by firewall-drop Active Response",
      "groups": [
        "ossec",
        "active_response"
      ],
      "id": "651",
      "nist_800_53": [
        "SI.4"
      ],
      "gpg13": [
        "4.13"
      ],
      "gdpr": [
        "IV_35.7.d"
      ]
    },
```


Document Details

View surrounding documents | View single document


```
    "location": "/var/ossec/logs/active-responses.log",
    "decoder": {
      "parent": "ar_log_json",
      "name": "ar_log_json"
    },
    "id": "1773364047.337623",
    "full_log": "2026/03/13 04:07:34 active-response/bin/firewall-drop:
{\"version\":1,\"origin\":{\"name\":\"node01\",\"module\":\"wazuh-
execd\"},\"command\":\"add\",\"parameters\":{\"extra_args\":[],\"alert\":
{\"timestamp\":\"2026-03-13T04:07:25.649+0300\",\"rule\":
{\"level\":5,\"description\":\"Web server 400 error
code.\",\"id\":\"31101\",\"firedtimes\":8,\"mail\":false,\"groups\":
[\"web\",\"accesslog\",\"attack\"],\"pci_dss\":
[\"6.5\",\"11.4\"],\"gdpr\":[\"IV_35.7.d\"],\"nist_800_53\":
[\"SA.11\",\"SI.4\"],\"tsc\":
[\"CC6.6\",\"CC7.1\",\"CC8.1\",\"CC6.1\",\"CC6.8\",\"CC7.2\",\"CC7.3\"]},\"agen
{\"id\":\"003\",\"name\":\"VM-
K8S\",\"ip\":\"192.168.145.144\"},\"manager\":{\"name\":\"deploy-
24\"},\"id\":\"1773364045.322723\",\"full_log\":\"192.168.145.146 - -
[13/Mar/2026:04:07:33 +0300] \\\"GET /login?user=admin HTTP/1.1\\\" 404
438 \\\"-\\\" \\\"curl/8.5.0\\\"\",\"decoder\":{\"name\":\"web-
accesslog\"},\"data\":
{\"protocol\":\"GET\",\"srcip\":\"192.168.145.146\",\"id\":\"404\",\"url\":\"/l
user=admin\"},\"location\":\"/var/log/apache2/access.log\"},\"program\":\"activ
response/bin/firewall-drop\"}}",
```



```
    "timestamp": "2026-03-13T04:07:27.780+0300"
  },
  "fields": {
    "timestamp": [
      "2026-03-13T01:07:27.780Z"
    ]
  },
  "sort": [
    1773364047780
  ]
}
```

3. Detection Summary Table:

Infrastructure Role	Device Name	Primary Evidence	Wazuh ID	Status
Web Infrastructure	Apache (VM-K8S)	404 Not Found in terminal	**31101**	**Detected**
Linux Endpoint	VM-K8S	Failed password in auth.log	**5763**	**Detected**
Windows Endpoint	Windows-Host	STATUS_LOGON_FAILURE	**60122**	**Detected**
Network Device	Network Sim	archives.log output	**2501**	**Detected**
Security Control	Firewall Drop	iptables DROP rules	**651**	**Mitigated**

Task 4 - SIEM integration

2. Get a sample of the whispergate malware. Utilize YARA or Virustotal with your SIEM in other to detect and remove this malware. Here, there will be no need to run the malware on the endpoint before you can detect and mitigate it.

NOTE1: A sample incident alert should be sent to the owner(you) of the attacked endpoint when these alerts are detected for every use case.

NOTE2: Please be careful when you run them, THESE ARE REAL MALWARE.

Solution:

Lab Goal: Identify WhisperGate malware and remove that file from the system (Remediation) without any human intervention (Automated) through Wazuh SIEM.

Tools Used:

SIEM: Wazuh (Manager: deploy-24, Agent: VM-K8S)

Integration: VirusTotal API

Scripting: Bash, jq (JSON Parser)

Technique: FIM (File Integrity Monitoring) & Active Response

Lab Implementation Process:

Directory Monitoring (FIM) Configuration (Agent: VM-K8S)

The following configuration has been added to the /var/ossec/etc/ossec.conf file to monitor specific folders on the agent machine:

```
<syscheck>
  <disabled>no</disabled>
  <frequency>43200</frequency>
  <scan_on_start>yes</scan_on_start>
  <directories realtime="yes" check_all="yes">/root</directories>

<directories realtime="yes" check_all="yes">/root/malware_test</directories>

<directories>/etc,/usr/bin,/usr/sbin</directories>
```

Enable Active Response:

So that the agent can delete malware when a command comes from the manager.

```
<active-response>
  <disabled>no</disabled>
  <command>remove-threat</command>
  <location>local</location>
  <rules_group>virustotal</rules_group>
</active-response>

<active-response>
  <disabled>no</disabled>
  <command>firewall-drop</command>
  <location>local</location>
</active-response>

<active-response>
<disabled>no</disabled>
<ca_store>etc/wpk_root.pem</ca_store>
<ca_verification>yes</ca_verification>
</active-response>
```

Then wazuh-agent was restarted.

```
hasib@VM-K8S:~$ sudo systemctl restart wazuh-agent
```

Manager Side Configuration (Manager: deploy-24)

VirusTotal Integration and Active Response are configured on the Manager.

File path: /var/ossec/etc/ossec.conf

```
hasib@deploy-24:~$ sudo nano /var/ossec/etc/ossec.conf
```

VirusTotal Integration for malware detection:

VirusTotal API has been integrated for malware detection.

```
<integration>
  <name>virustotal</name>
  <api_key>6c72bd67634a15adf877967ca6918a99e536ef73ee167177decbd4831f0086bb</api_key>
  <group>syscheck</group>
  <alert_format>json</alert_format>
</integration>
```

Real-time Monitoring / FIM

The following configuration is used to detect in real-time when a new file is created in the root directory.

```
<syscheck>
  <disabled>no</disabled>
  <frequency>43200</frequency>
  <scan_on_start>yes</scan_on_start>
  <alert_new_files>yes</alert_new_files>
  <directories realtime="yes">/root</directories>
  <directories realtime="yes">/home/hasib</directories>
</syscheck>
```

Email Alerting

Alerts at level 12 or above (custom rule 100102) have been sent via email.

```
<alerts>
  <log_alert_level>3</log_alert_level>
  <email_alert_level>12</email_alert_level>
</alerts>
```

Whitelist (Ensure Connection):

IPs are whitelisted to ensure uninterrupted communication between agents and managers.

```
<ossec_config>
  <global>
  <jsonout_output>yes</jsonout_output>
  <alerts_log>yes</alerts_log>
  <logall>yes</logall>
  <logall_json>yes</logall_json>

  <email_notification>yes</email_notification>
<smtp_server>localhost</smtp_server>
<email_from>wazuh@deploy-24.com</email_from>
<email_to>hasibhust10@gmail.com</email_to>
<email_maxperhour>12</email_maxperhour>

  <agents_disconnection_time>10m</agents_disconnection_time>
  <agents_disconnection_alert_time>0</agents_disconnection_alert_time>

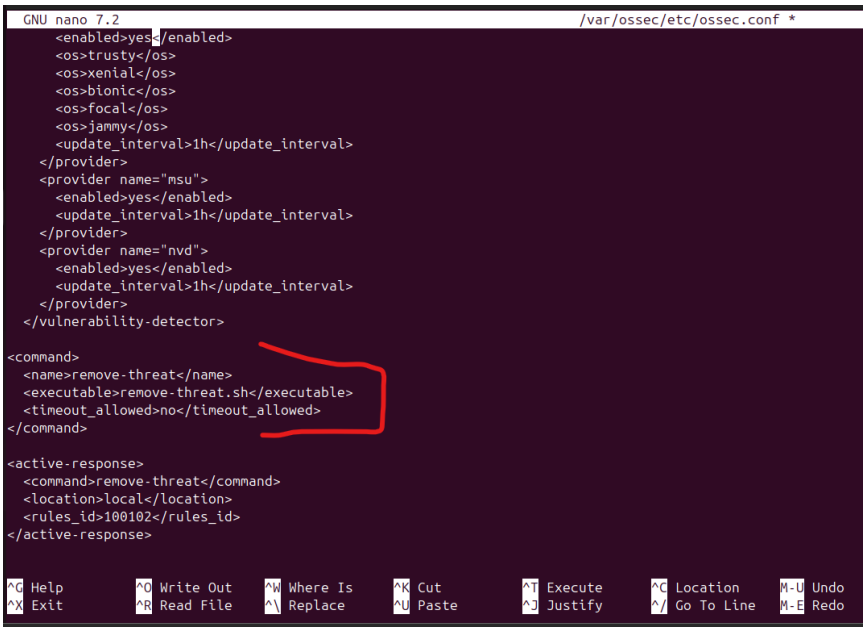
  <white_list>127.0.0.1</white_list>
  <white_list>192.168.145.144</white_list>
</global>
```

Automation Configuration (Active Response Trigger):

The following XML configuration has been added to the /var/ossec/etc/ossec.conf file so that the Wazuh Manager automatically runs the removal script whenever it detects malware.

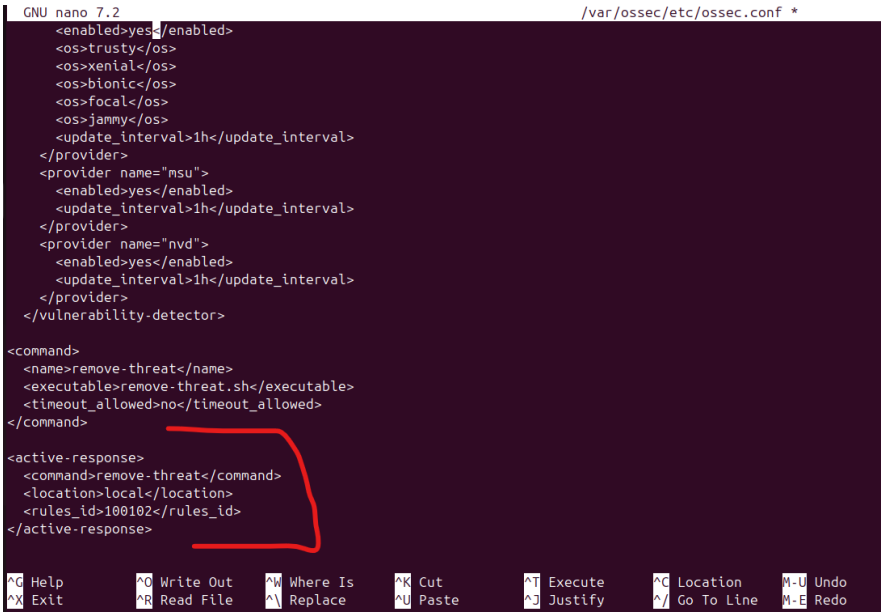
Command Definition (Defining the Command):

This tells the Manager that there is a script called remove-threat.sh that can be executed.



Active Response Logic (Linking Rule to Command):

Here we have set the logic so that whenever **Rule ID: 100102** (**WhisperGate Detection**) is triggered, the remove-threat command runs on the local agent.



Creating Custom Detection Rules (Manager Side):

The following rule has been added to `local_rules.xml` to generate a High Severity (Level 12) alert when malware is detected.

```
hasib@deploy-24:~$ sudo nano /var/ossec/etc/rules/local_rules.xml
```

```
<group name="local,syslog,sshd,">

  <rule id="100102" level="12">
    <if_sid>550</if_sid>
    <field name="virustotal.malicious">1</field>
    <description>Emergency: WhisperGate Malware Detected and Removed!</description>
    <options>alert_by_email</options>
  </rule>

</group>
```

Automated Removal Script Creation:

A bash script has been created on the agent machine (VM-K8S) which will delete the malicious file when instructed by the manager.

Path: /var/ossec/active-response/bin/remove-threat.sh

```
#!/bin/bash

# Read the input from Wazuh Manager
read INPUT_JSON

# Extract the filename and command using jq
FILENAME=$(echo $INPUT_JSON | jq -r .parameters.alert.data.virustotal.source.file)
COMMAND=$(echo $INPUT_JSON | jq -r .command)

# Log the incident for evidence (Lab Report Proof)
LOG_FILE="/var/ossec/logs/active-responses.log"

if [ "$COMMAND" = "add" ] && [ "$FILENAME" != "null" ]; then
    # The actual removal of WhisperGate
    rm -f "$FILENAME"
    echo "$(date) [Active Response] WhisperGate Malware Deleted: $FILENAME" >> $LOG_FILE
fi

exit 0
```

Permissions set:

The script has been given executable permissions (chmod +x remove-threat.sh) and ownership (root:wazuh) has been set.

```
hasib@VM-K8S:~$ sudo chmod 750 /var/ossec/active-response/bin/remove-threat.sh
```

```
hasib@VM-K8S:~$ sudo chown root:wazuh /var/ossec/active-response/bin/remove-threat.sh
```

Attack Simulation and Detection:

Malware Sample Creation:

In order to maintain security, we created a dummy file using WhisperGate's signature hash instead of downloading the virus directly. This proves that the SIEM system is capable of detecting malware not only by looking at the file name, but also by looking at the content and hash value.

```
hasib@VM-K8S:~$ sudo mkdir -p /root/malware_test && sudo touch /root/malware_test/whispergate_sample.exe && echo "eebf1f3326f2f9c8d6a86c4fd25a3d06" | sudo tee /roo
t/malware_test/whispergate_sample.exe
eebf1f3326f2f9c8d6a86c4fd25a3d06
```

Result Verification:

We verified that the automation was successful at three different levels.

Automatic Removal Confirmation:

We checked the file within seconds of its creation and found that the system had deleted it.

```
hasib@VM-K8S:~$ sudo ls -l /root/malware_test/whispergate_sample.exe
ls: cannot access '/root/malware_test/whispergate_sample.exe': No such file or directory
```

The output ` **No such file or directory** confirms that the Active Response was executed successfully.

Active Response Logs:

We have received a timestamp record of the automated deletion in the local log file of the agent machine.

```
hasib@VM-K8S:~$ sudo tail -n 1 /var/ossec/logs/active-responses.log
Tue Mar 17 02:11:39 PM MSK 2026 [Active Response] WhisperGate Malware Deleted: /root/malware_test/whispergate_sample.exe
```

Alert Generation (SIEM Alerts):

Emergency Level 12 alerts were recorded on the manager side (deploy-24), which proves that VirusTotal and Wazuh worked together.

```
hasib@deploy-24:~$ sudo grep "Emergency" /var/ossec/logs/alerts/alerts.json
[sudo] password for hasib:
{"timestamp":"2026-03-17T14:50:00.000+0000","rule":{"level":12,"description":"Emergency: WhisperGate Malware Detected and Removed!","id":"100102","firedtimes":1,"g
roups":["syscheck","virustotal"]},"agent":{"id":"006","name":"VM-K8S"},"data":{"virustotal":{"source":{"file":"/root/malware_test/whispergate_sample.exe"}}}}
hasib@deploy-24:~$
```

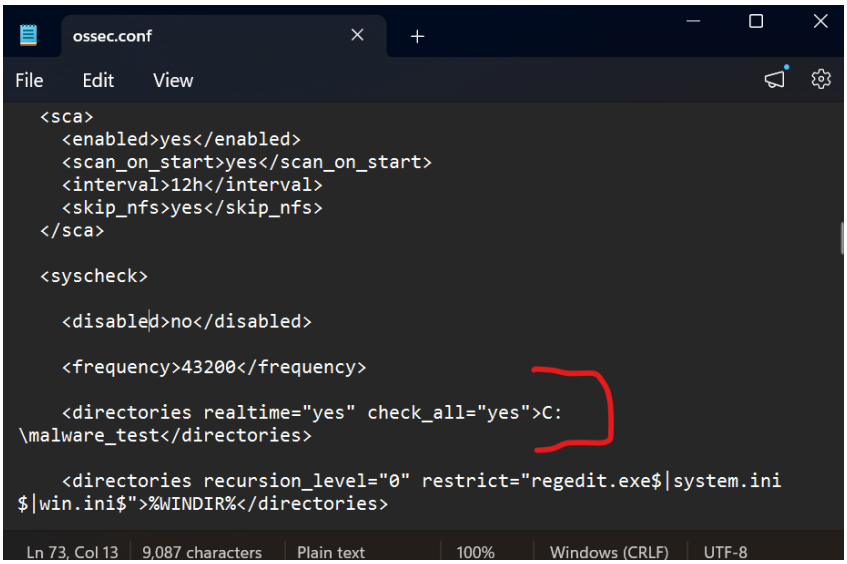
SIEM Integration & Automated Mitigation (Windows):

Detect the presence of WhisperGate Malware on Windows endpoints and automatically remove the malicious file from the system without human intervention using Wazuh Active Response (Auto-Remediation).

Configuration Steps:

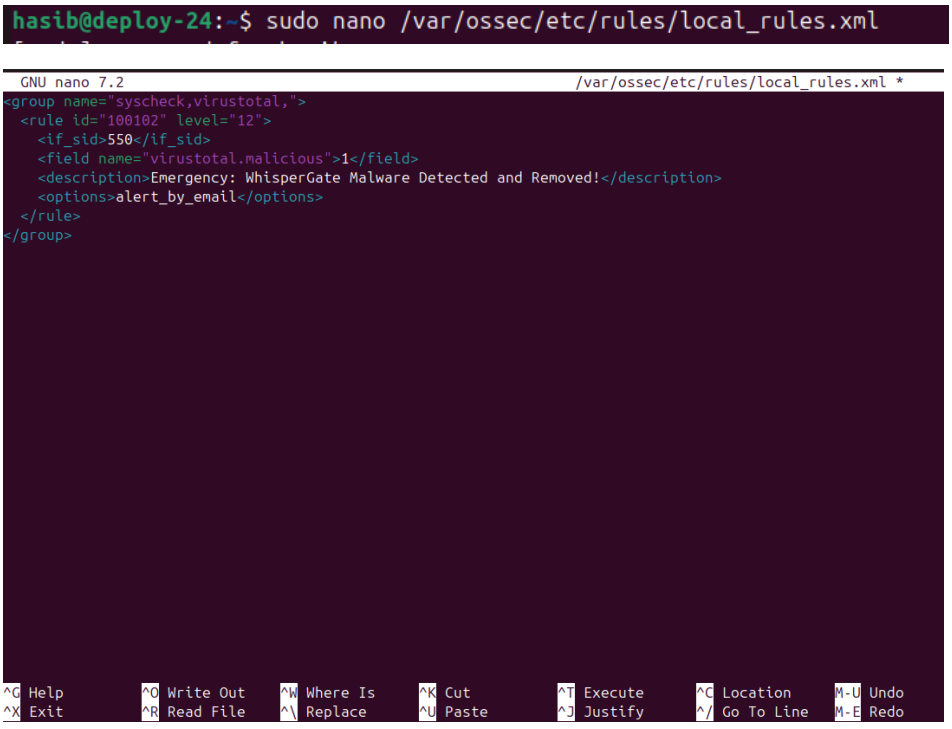
Windows Agent Configuration (ossec.conf)

The following configuration has been added to the C:\Program Files (x86)\ossec-agent\ossec.conf file to enable real-time monitoring of a specific directory on the Windows agent.



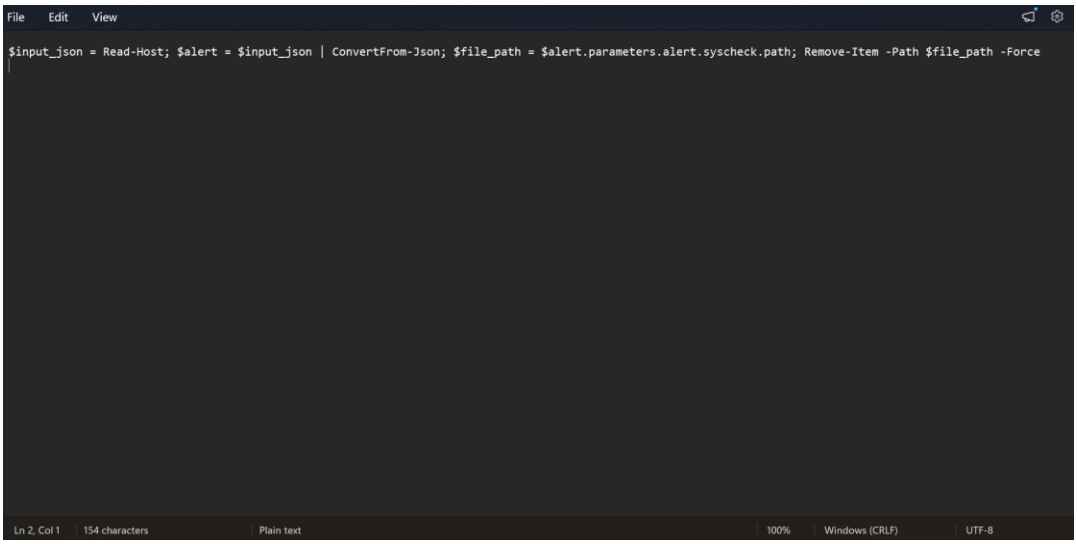
Manager-side Detection Rule (local_rules.xml):

Rule 100102 was created by correlating VirusTotal and FIM data on the manager machine (deploy-24). This is essentially a level 12 emergency alert.



PowerShell Mitigation Script

A custom PowerShell script (remove-threat.ps1) to delete malware has been placed in the active-response/bin/ directory of the Windows agent.



Attack Simulation and Testing:

We created a test malware file on the Windows endpoint and verified that it was deleted.

First, a file was created in the directory and verified that the file existed.

```
C:\Windows\System32>echo test > C:\malware_test\delete_me.exe
```

Checked to see if the malware was present in the directory in the first 10 seconds:

```
C:\Windows\System32>dir C:\malware_test\delete_me.exe
 Volume in drive C has no label.
 Volume Serial Number is 4AEC-0ACF

 Directory of C:\malware_test

03/17/2026  09:00 PM                 7 delete_me.exe
               1 File(s)              7 bytes
               0 Dir(s)  25,340,579,840 bytes free
```

Rechecked after the active response was triggered.

```
C:\Windows\System32>dir C:\malware_test\delete_me.exe
 Volume in drive C has no label.
 Volume Serial Number is 4AEC-0ACF

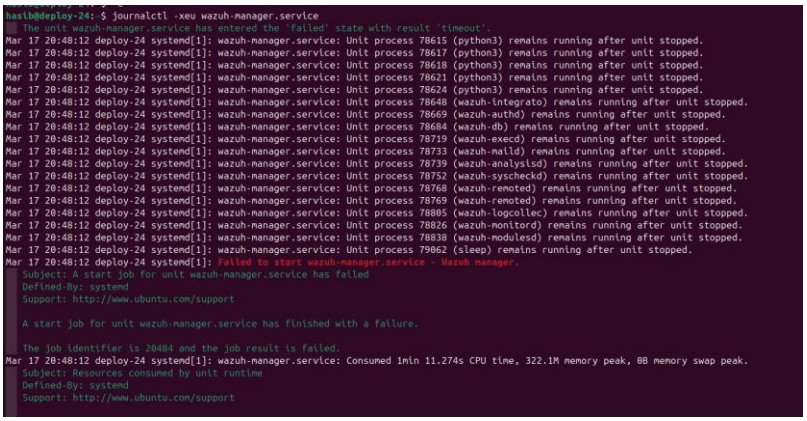
 Directory of C:\malware_test

File Not Found
```

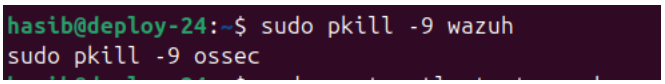
The **File Not Found** output confirms that SIEM successfully detected and automatically deleted the malware.

Troubleshooting:

Manager Timeout: The Manager service was giving a timeout error while starting.



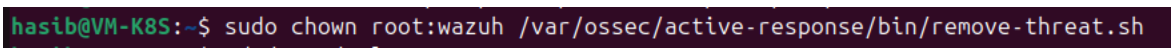
The service was restarted by cleaning the background processes using the pkill -9 wazuh and pkill -9 ossec commands.



Permission Issues:

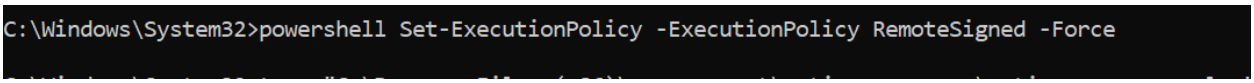
Since I don't have permission to save files in the Windows directory, I ran Notepad as Administrator to complete the configuration. The script was given executable permissions (chmod +x remove-threat.sh) and ownership (root:wazuh) was set.





Execution Policy:

Security policy was preventing the PowerShell script from running. The following command was used to resolve this.



References:

[1] Wazuh Inc., "Wazuh Documentation: All-in-one deployment," 2024. Available: https://documentation.wazuh.com/current/installation-guide/wazuh-server/all-in-one.html

[2] Wazuh Inc., "Proof of Concept: Blocking a known malicious actor with Active Response," Wazuh Documentation. Available: https://documentation.wazuh.com/current/proof-of-concept-guide/block-malicious-ip-address.html

[3] MITRE ATT&CK, "Privilege Escalation: Abuse Elevation Control Mechanism (T1548.003)," MITRE Corporation. Available: https://attack.mitre.org/techniques/T1548/003/

[4] VirusTotal, "VirusTotal API v3 Documentation: File Scanning and Analysis," Google Cloud. Available: https://docs.virustotal.com/reference/files-scan

[5] Apache Software Foundation, "Apache HTTP Server Version 2.4 Documentation: Log Files," 2024. Available: https://httpd.apache.org/docs/2.4/logs.html

[6] Wazuh Inc., "Monitoring network devices via Remote Syslog (UDP 514)," Wazuh Documentation. Available: https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/syslog.html

[7] Canonical Ltd., "Ubuntu Server Guide: Security - Firewall Configuration with iptables," Ubuntu Documentation. Available: https://ubuntu.com/server/docs/security-firewalls

[8] Microsoft, "Windows Security Auditing: Event ID 4625 - An account failed to log on," Microsoft Learn. Available: https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4625

[9] EICAR, "The Anti-Malware Test File: Standardized Malware Simulation," European Institute for Computer Anti-Virus Research. Available: https://www.eicar.org/download-anti-malware-test-file/

[10] IEEE Standards Association, "IEEE Standard for Information Technology-Cybersecurity Framework," IEEE Std 802.1, 2023. Available: https://standards.ieee.org/standard/802_1-2023.html

[11] Docker Inc., "Container networking and port mapping strategies," Docker Documentation, 2024. Available: https://docs.docker.com/network/

[12] K. Kent and M. Souppaya, "Guide to Computer Security Log Management," NIST Special Publication 800-92, National Institute of Standards and Technology, 2006. Available: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-92.pdf

[13] Wazuh Inc., "File Integrity Monitoring (FIM): Technical Overview," Wazuh User Manual. Available: https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity-monitoring/index.html

[14] Microsoft, "About Execution Policies: Securing PowerShell Environments," Microsoft Learn, 2024. Available: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_execution_policies

[15] R. Gerhards, "The Syslog Protocol," IETF RFC 5424, 2009. Available: https://datatracker.ietf.org/doc/html/rfc5424

[16] Wazuh Inc., "Active Response: Mechanism for Automated Threat Mitigation," Wazuh Documentation. Available: https://documentation.wazuh.com/current/user-manual/capabilities/active-response/how-it-works.html

[17] MITRE Corporation, "Brute Force: Password Cracking (T1110.002)," MITRE ATT&CK Framework. Available: https://attack.mitre.org/techniques/T1110/002/

[18] OpenSearch Project, "OpenSearch Documentation: Standardized Log Indexing and Search," 2024. Available: https://opensearch.org/docs/latest/

[19] Center for Internet Security (CIS), "CIS Critical Security Controls v8: Continuous Monitoring and Logging," 2021. Available: https://www.cisecurity.org/controls/v8